Leveraging AI for Enhancing Software Development Practices Through Collaborative Computing: A systematic Mapping Study

COMP971 DISSERTATION

Principal Programme: AK1329, Master of Computer and Information Sciences

ROHAN HOLEGADDE SURESH

Student ID: 24227139

Supervisor: Tony Clear

Associate Professor

Engineering, Computer and Mathematical Sciences

ABSTRACT

The emergence of AI-driven tools such as GitHub Copilot, Amazon Q Developer, and ChatGPT has sparked a paradigm shift in software engineering, particularly within collaborative development environments. This dissertation presents a systematic mapping study that investigates the impact of AI tools and techniques on collaborative software development practices, focusing on their influence on team productivity, coordination, communication, and software quality. The study addresses one primary research question, examining the overarching impact of AI on collaboration, and four sub-questions that delve into AI-assisted pair programming, debugging, software quality assessment, and the identification of adoption gaps.

Following a systematic and replicable methodology, this study analyzed 55 peer-reviewed publications sourced from major academic databases using a defined classification scheme. The findings reveal a growing adoption of AI tools in practices such as code generation, onboarding, documentation, and DevSecOps integration. GitHub Copilot and Amazon Q Developer emerged as the most frequently studied tools, often associated with increased development speed and improved task efficiency. However, the literature also reveals significant gaps, particularly in evaluating long-term effects on maintainability, ethical and legal implications of AI-generated code, and the lack of team-level performance metrics.

The study concludes that while AI tools offer substantial benefits in enhancing individual productivity and streamlining routine tasks, their integration into collaborative workflows remains uneven and under-theorized. Practical recommendations are proposed for developers, team leads, DevOps engineers, and tool vendors, emphasizing responsible adoption, lifecycle governance, and training for effective AI-human collaboration. This research contributes a structured overview of the field and identifies key directions for future inquiry, offering a foundational reference for both academics and practitioners seeking to navigate the evolving intersection of artificial intelligence and collaborative software engineering.

Keywords: *Artificial Intelligence, Collaborative Software Development, AI tools Systematic Mapping Study, GitHub Copilot, Amazon Q Developer, Software Engineering, Team Productivity, Pair programming*

Acknowledgements

I would like to express my heartfelt appreciation to my supervisor, **Professor Tony Clear**, for his exceptional guidance, mentorship, and unwavering support throughout the course of this dissertation. His expertise in software engineering and critical thinking not only enriched the academic quality of this work but also challenged me to approach research with greater depth, clarity, and reflection.

Professor Clear's insightful feedback, patient encouragement, and constructive critique were instrumental at every stage, from the formulation of the research questions to the final synthesis of findings. His commitment to fostering independent thought and scholarly rigor has significantly shaped my growth as a researcher.

I am deeply grateful for the opportunity from Auckland University of Technology and to have worked under his supervision and for the inspiration he has provided throughout this journey.

Contents

CHAPTER 1: INTRODUCTION 4

1.1 Background and Context 4
1.2 Problem statement 5
1.3 Aim and Objectives 6
1.4 Motivation for the Research / Significance of the Study 7
1.5 Structure of Dissertation 7
CHAPTER 2: LITERATURE REVIEW 8
2.1 Introduction to Collaborative Computing 8
2.2 AI in Software Development 11
2.3 AI for Collaboration Enhancement 14
2.4 Research Gap 20
CHAPTER 3: RESEARCH METHODLOGY 22
3.1 Research Approach 22
3.2 Research Questions 23
3.3 Search Strategy 25
3.4 Selection and Screening Process [PRISMA Diagram] 28
3.5 Classification Scheme 29
3.6 Data Extraction and Mapping of Studies 31
CHAPTER 4: RESULTS 35
4.1 Overview of Included Studies 35
4.2 Mapping Results 36
4.3 Findings Addressing Each Research Question Explicitly 38
CHAPTER 5: DISCUSSION 40
5.1 Interpretation of Results 40
5.2 Implications for Software Development Practice 41
5.3 Limitations 44
5.4 Answers to the Research Questions 44
5.5 Recommendations for Future Research 45
CHAPTER 6: CONCLUSION 46
REFERENCES 48
APPENDIX 51
Appendix A: Evidence of Research Depth and Critical Engagement 51
Appendix B: List of Figures and Tables 53

CHAPTER 1: INTRODUCTION

1.1 Background and Context

1.1.1 Importance of Collaborative Computing in Software Development

Software development today is a highly collaborative endeavor. Modern software projects typically involve teams that are geographically distributed and often multidisciplinary, requiring seamless coordination and communication to ensure project success. Collaborative computing environments facilitate this by enabling distributed teams to work together efficiently, sharing resources, knowledge, and responsibilities.

According to Herbsleb and Mockus (Herbsleb & Mockus, 2003), even small increases in physical distance between developers can significantly reduce the frequency of communication, highlighting the necessity for robust collaborative systems. The complexity of modern software, coupled with short development cycles driven by agile and DevOps methodologies, has intensified the need for effective collaboration platforms that can support real-time communication, task sharing, version control, continuous integration, and conflict resolution.

Collaborative computing not only enhances productivity but also directly impacts software quality. Tools that enable better collaboration help mitigate risks like code conflicts, inconsistent documentation, and misaligned project goals, thereby improving the maintainability, scalability, and overall success of software products (Herbsleb & Mockus, 2003). In an era of globally distributed development teams, the importance of technologies that foster collaboration has become more critical than ever (Cui et al., 2024).

1.1.2 The Rise of AI-Powered Tools in Collaborative Software Development

In recent years, artificial intelligence (AI) has emerged as a transformative force in software engineering, ushering in a new era of intelligent collaborative computing. AI-powered coding assistants are now deeply embedded within development workflows, supporting tasks such as coding, debugging, testing, documentation, and maintenance. These tools not only automate repetitive or error-prone operations but also augment developers' cognitive capabilities, allowing them to concentrate on higher-order problem-solving and innovation.

A notable milestone in this evolution was the launch of GitHub Copilot in June 2021, powered by OpenAI’s Codex model. Integrated into IDEs, Copilot offers real-time, context-aware code suggestions, function completions, and documentation generation. Surveys report that 73% of users feel Copilot helps preserve their mental energy, while studies show productivity gains of up to 55% in task completion and up to 21.8% more pull requests per week (Vaithilingam et al., 2022) (Song et al., 2024)

Following suit, Amazon introduced CodeWhisperer in June 2022 and released it publicly in April 2023. By 2024, it had evolved into Amazon Q Developer, a more comprehensive AI assistant that goes beyond code completion. In addition to generating code, Amazon Q Developer scans for security vulnerabilities, suggests remediations, and integrates with AWS services to streamline real-time issue resolution across distributed development environments (Lakhwani, 2024). It supports popular IDEs such as VS Code, IntelliJ, and Visual Studio, offering end-to-end assistance from development to deployment.

These tools are particularly impactful in collaborative development contexts. They promote consistency, reduce manual review overhead, and embed security best practices directly into

the coding process. Early evidence also indicates that they enhance code quality and reduce defect rates, leading to more secure, resilient, and high-velocity software development.

Ultimately, the integration of AI into software development marks a paradigm shift from passive tool usage to active human-AI collaboration, fundamentally redefining how software is designed, built, and maintained.

1.2 Problem statement

1.2.1 Identification of Gaps in Current Practices

Despite the rapid integration of AI-powered coding assistants such as GitHub Copilot and Amazon Q Developer into modern development workflows, significant gaps persist in understanding their impact on collaborative software engineering practices. While these tools have demonstrated value in automating code generation, documentation, and vulnerability detection, their broader implications on team dynamics, communication efficiency, and secure collaborative development remain underexplored.

Current research has largely adopted an individual-centric focus. Controlled studies often highlight productivity gains, for instance, developers complete up to 21.8% more pull requests per week using GitHub Copilot (Song et al., 2024) (Cui et al., 2024) but offer limited insights into how these tools affect collective practices such as team coordination, peer review processes, conflict resolution, and shared code ownership. This narrow focus constrains our understanding of how AI tools reshape collaboration in distributed and large-scale environments.

Additional gaps in the literature include:

- **Team-Level Impact Remains Understudied**: Most evaluations overlook team-level metrics, such as communication effectiveness, joint decision-making, and collaborative problem-solving outcomes. As a result, there is insufficient evidence on how AI tools influence group cohesion and productivity in real-world projects.
- **Mixed Evidence on Effectiveness and Code Quality**: While some studies report productivity boosts, others have observed increased defect rates, up to 41% in some cases, raising questions about the reliability and quality implications of AI-generated code in collaborative contexts (Pearce et al., 2022).
- **Integration Challenges in Heterogeneous Environments**: There is a lack of empirical evidence regarding the seamless integration of AI assistants into diverse toolchains involving multiple IDEs, version control systems, and CI/CD pipelines. This is particularly critical for organizations with complex, distributed workflows (Liang et al., 2024).
- **Security and Ethical Considerations in Team Settings**: Tools like Amazon Q Developer include built-in security scanning, yet their effectiveness in fostering secure collaborative coding practices, especially during shared debugging and code reviews, has not been rigorously studied (Fu et al., 2025) (Ajiga et al., 2024).
- **Lack of Longitudinal Studies**: Most insights are derived from short-term evaluations. The long-term impact of AI-assisted development on software quality, developer satisfaction, and sustained team performance across full project lifecycles is still largely unknown (Seppälä, 2024).

Furthermore, existing systematic mapping studies tend to examine isolated AI techniques or narrow lifecycle stages, without comprehensively addressing AI's role in supporting end-to-

end collaborative computing environments (Seppälä, 2024). Addressing these research gaps is essential for realizing the full potential of AI in enhancing not only individual productivity but also team-based software engineering outcomes (Nguyen-Duc et al., 2023).

1.2.2 Justification of Research Area Selection

Addressing the identified gaps is essential, as collaborative computing forms the foundation of modern software engineering particularly in agile driven, globally distributed development environments. As organizations increasingly adopt AI-powered tools like GitHub Copilot and Amazon Q Developer, there is a pressing need to understand their influence not just on individual productivity, but also on team-level dynamics, communication, and secure software practices (Yetiştiren et al., 2023).

This research area is justified by several critical factors:

- **Strategic Relevance for Distributed Teams**: In distributed development settings, even small physical distances between team members can significantly reduce communication frequency, delaying integration and coordination (Herbsleb & Mockus, 2003). AI tools have the potential to bridge these gaps by facilitating more efficient collaboration, real-time feedback, and task automation across geographies.
- **Transformative Potential of Next-Generation AI Tools**: The evolution from focused assistants like CodeWhisperer to comprehensive platforms such as Amazon Q Developer signals a paradigm shift (Yetiştiren et al., 2023). These tools now span multiple development tasks; coding, debugging, documentation, and security, warranting a holistic assessment of their impact on team workflows and collaborative efficiency.
- **Academic and Industry Relevance**: While prior studies have explored AI in pair programming or isolated development phases (Friedmann, 2024) (Ganeshan, 2024), few have mapped the intersection of AI adoption and collaborative software engineering practices. A systematic mapping study focused on this nexus will contribute significantly to the academic literature and provide actionable insights for industry stakeholders.
- **Guidance for Tool Design and Future Research**: By identifying underexplored areas such as the long-term effects of AI tools on team cohesion, shared code ownership, and collaborative security practices, this study will offer a roadmap for future empirical research. It will also inform the design of AI-assisted environments that go beyond individual task automation to foster deeper collaboration and resilience in software teams.

In summary, this study serves both practical and scholarly objectives. It aims to advance the understanding of AI's role in team-based software development, supporting the development of intelligent systems that not only enhance productivity but also elevate the collaborative fabric of modern software engineering teams.

1.3 Aim and Objectives

1.3.1 Overall Research Aim

The primary aim of this study is to systematically map and critically analyse the existing body of literature on AI-driven tools in collaborative software development. The focus is to evaluate how these tools influence team productivity, communication, coordination mechanisms, and software quality particularly within geographically distributed and agile-

driven environments. By doing so, the study seeks to bridge the gap between individual-centric evaluations of AI tools and their broader, team-level impacts.

1.3.2 Specific Research Objectives

To achieve the research aim, this mapping study is guided by the following objectives:

1. **Identification and Classification**
   Identify and classify the selected AI tools and techniques currently employed in collaborative software engineering, using established mapping frameworks to organize findings by development phase, functionality, and collaboration support.
2. **Impact Evaluation**
   Evaluate the effects of AI-assisted systems, such as GitHub Copilot and Amazon Q Developer, on collaborative activities including conflict resolution, code review processes, joint debugging, and shared code ownership.
3. **Assessment of Benefits and Limitations**
   Assess the advantages and challenges associated with the adoption of the above-mentioned AI tools in distributed development teams, particularly in terms of team productivity, software quality, and developer satisfaction.
4. **Exploration of Integration, Security, and Ethical Challenges**
   Investigate the practical risks and barriers related to embedding AI tools within collaborative toolchains, continuous integration pipelines, and multi-platform environments, including concerns around security, bias, and ethical use.
5. **Research Gap Identification and Future Directions**
   Highlight underexplored areas in the current literature and propose a set of future research directions aimed at optimizing AI-supported collaborative development practices for both academic inquiry and industry application.

1.4 Motivation for the Research / Significance of the Study

Theoretical Implications: This research contributes to the evolving field of software engineering by extending systematic mapping methodologies to the emergent intersection of AI technologies and team-based development practices. It enriches socio-technical theory by moving beyond individual performance metrics to explore how AI shapes group dynamics, communication flow, and cognitive collaboration within distributed teams. The structured classification of tools, techniques, and study designs can serve as a foundational reference for future empirical research and meta-analyses.

Practical Implications: For industry stakeholders, the findings offer actionable guidance on the effective selection, integration, and governance of AI tools within collaborative workflows. The study synthesizes real-world insights on the usability, contextual applicability, and limitations of tools like GitHub Copilot and Amazon Q Developer, aiding software organizations, team leads, and platform providers in making informed decisions. By addressing practical concerns such as security, workflow disruption, and developer experience, the research aims to support the design of more intelligent, secure, and scalable AI-assisted software development environments.

1.5 Structure of Dissertation

This dissertation is organized into six chapters. Chapter 2 presents a comprehensive **literature review**, beginning with an introduction to collaborative computing in software development, emphasizing its growing relevance in distributed environments. It then explores

the emergence of AI technologies and their current applications in software engineering, particularly in enhancing collaborative practices. The chapter concludes by identifying significant research gaps, laying the groundwork for the present study.

Chapter 3 outlines the **research methodology** employed, detailing the rationale for selecting a systematic mapping study approach. It presents the research questions that guide the investigation and describes the search strategy, selection and screening process (including a PRISMA diagram), and the data extraction and classification scheme. Considerations of quality assessment and strategies for ensuring validity and reliability are also discussed to strengthen the methodological rigor.

Chapter 4 presents the **results** of the mapping study, including descriptive statistics of the selected literature and thematic categorizations that reveal patterns in AI tool use and collaboration outcomes. Chapter 5 offers a critical **discussion** of these findings, interpreting their significance in relation to the research questions, comparing them with existing work, and outlining practical implications for industry and academia. Answering the research questions along with the limitations of this mapping study process and also proposing directions for future research. The final Chapter 6, delivers the **conclusion**, restating the research aims and questions, summarizing the key contributions of the study by acknowledging its limitations, and reflecting on research significance and contributions to knowledge.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction to Collaborative Computing

2.1.1 Definition and Evolution

Collaborative computing in software development refers to the use of digital tools, platforms, and processes that enable multiple developers to work together synchronously or asynchronously on shared codebases (Hildenbrand et al., 2008). This concept emerged as a response to the increasingly distributed nature of software teams and the growing complexity of development tasks. Collaborative computing encompasses mechanisms such as version control systems (e.g., Git), continuous integration/continuous deployment (CI/CD), code review systems, and team communication tools like Slack and Microsoft Teams (Rautiainen, 2023).

What is a Collaborative development environment? It is a virtual workspace where all the project stakeholders, regardless of their geographical location or time zone, can come together to negotiate, brainstorm, communicate, share knowledge, and collectively work towards completing tasks, typically aimed at producing an executable product along with its related documentation and artifacts (Booch & Brown, 2003).

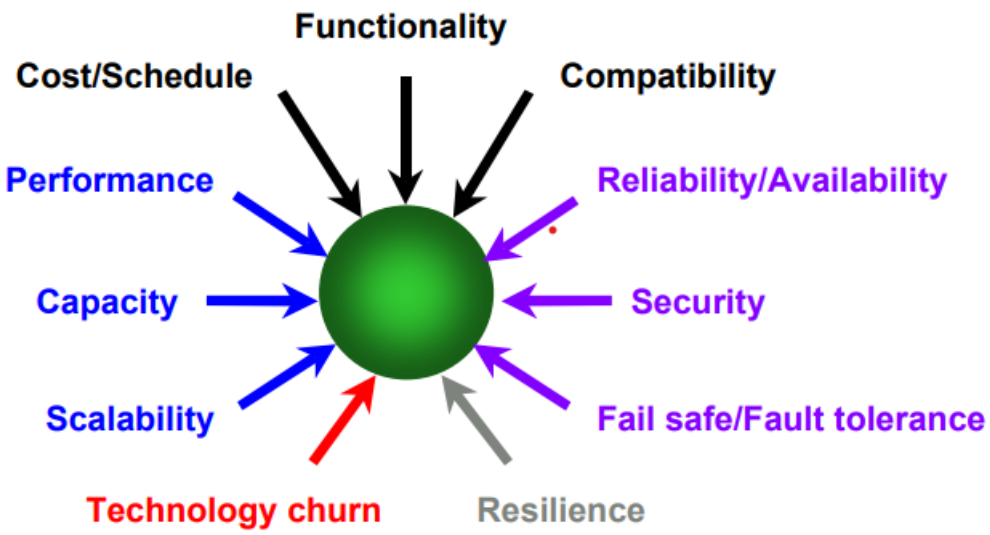

Figure 1: Forces of software by Booch, G., & Brown, A. W. (2003). Collaborative development environments. Adv. Comput., 59(1), 1-27 (Booch & Brown, 2003)

Figure 1 depicts the multifaceted pressures facing software development teams. The most critical constraints include cost, schedule, functionality, and compatibility, particularly with legacy systems and third-party software (e.g., operating systems, middleware), which lie outside the team's direct control. Additional domain-specific challenges, such as performance, capacity, scalability, reliability, availability, security, and fault tolerance, may also influence priorities depending on the project's requirements (Booch & Brown, 2003).

Two external forces further complicate these dynamics:

1. Technology churn: Rapid evolution of hardware, protocols, and off-the-shelf software, over which teams have limited influence beyond initial adoption decisions.
2. Resilience: The necessity to design systems capable of adapting to continuous, heterogeneous change, where components evolve at differing paces.

This framework highlights how development teams must balance controllable factors with external dependencies and evolving technical landscapes (Booch & Brown, 2003).

The roots of collaborative software engineering trace back to the rise of distributed software projects in the late 1990s and early 2000s. Notably, Herbsleb and Mockus (Herbsleb & Mockus, 2003) documented how even teams located just a few buildings apart could suffer from coordination breakdowns, reduced communication, and slower integration, compared to co-located teams. Their findings highlighted the need for robust tools and workflows that support shared understanding, awareness of dependencies, and streamlined coordination in a geographically dispersed context.

Over time, collaborative development practices have matured through agile methodologies, which emphasize frequent communication, shared ownership, and iterative delivery. Practices like pair programming, daily stand-ups, and code reviews have become integral to fostering collaboration. Platforms like GitHub, GitLab, and Bitbucket have further evolved this paradigm by embedding social coding features, such as pull requests, commenting, and issue tracking, into the development lifecycle (Hildenbrand et al., 2008).

Recent Trends and Technological Advancements

The last decade has witnessed a surge in cloud-based development environments, such as GitHub Codespaces and AWS Cloud9, which enable developers to collaborate in real time from anywhere in the world. These platforms provide instant access to standardized development environments, reducing setup friction and promoting consistency across teams (Yetiştiren et al., 2023).

Another significant advancement is the integration of artificial intelligence (AI) and machine learning (ML) into collaborative workflows. AI-powered tools like GitHub Copilot, Amazon Q Developer, and Tabnine assist with code generation, bug detection, and documentation, further accelerating team productivity and reducing cognitive load (Yetiştiren et al., 2023) (Szolderits, 2025). These tools can also facilitate onboarding by providing contextual suggestions and explanations, lowering the barrier for new contributors to join ongoing projects.

The Role of DevOps and Automation

DevOps is a way of working that combines software development and IT operations to improve communication and make software updates happen more often, without losing quality. The word "DevOps" was created by Patrick Debois at a conference in 2009. It was developed to fix some problems with agile development, which was good at quickly creating code but not as fast at getting that code deployed (Moeez et al., 2024).

DevOps practices have become central to collaborative computing, bridging the gap between development and operations through automation, monitoring, and feedback loops. Automated testing, deployment pipelines, and infrastructure-as-code (IaC) allow teams to deliver features more reliably and efficiently, while minimizing manual intervention and reducing the risk of integration errors (Moeez et al., 2024).

Social and Organizational Dimensions

Collaborative computing is not solely a technical endeavour, it also encompasses the social dynamics of teamwork. Research has shown that psychological safety, shared mental models, and transparent communication are critical to the success of distributed teams (Edmondson, 2018). Edmondson’s work demonstrates that when team members feel safe to speak up and share ideas without fear of negative consequences, collaboration and team effectiveness are significantly enhanced (Edmondson, 2018). Modern tools increasingly support these needs by offering features like real-time chat, video conferencing, and collaborative editing, which help maintain alignment and foster a sense of community despite physical separation.

2.1.2 Challenges and Opportunities

While the evolution of collaborative computing has significantly transformed the way software is developed, it continues to present a complex landscape of challenges and opportunities. One of the most persistent obstacles is the coordination of work across different time zones, which can slow down communication and make synchronous collaboration difficult. Teams must often rely on asynchronous methods, such as detailed documentation and issue tracking, to ensure that progress continues smoothly even when members are not online at the same time. Additionally, cultural diversity within global teams brings both richness and complexity; differences in communication styles, work habits, and expectations can sometimes lead to misunderstandings or conflict if not managed thoughtfully. Varying levels of technical expertise among team members can also create

disparities in participation and productivity, making it essential for organizations to invest in continuous learning and effective onboarding processes (Mistrík et al., 2010).

Security and privacy concerns are heightened in distributed environments, where sensitive code and data are accessed from multiple locations and devices. This necessitates the implementation of robust access controls, authentication mechanisms, and audit trails to protect intellectual property and maintain compliance with regulatory standards. The risk of unauthorized access or data breaches is ever-present, requiring teams to be vigilant and proactive in their security practices (Mistrík et al., 2010).

Despite these challenges, the field of collaborative computing is ripe with opportunities. Advances in cloud-based development environments, AI-powered coding assistants, and integrated communication platforms are making it easier than ever for diverse teams to work together seamlessly. These tools not only enhance productivity but also foster greater inclusivity by enabling participation from individuals who might previously have been excluded due to geographic or logistical barriers. Furthermore, the scalability offered by modern collaborative platforms allows organizations to rapidly assemble and reconfigure teams to meet changing project demands, driving innovation and agility in software development. As technology continues to advance, the potential for collaborative computing to enable more creative, efficient, and inclusive approaches to building software will only continue to grow (Mistrík et al., 2010).

2.2 AI in Software Development

2.2.1 Overview of AI Technologies

Artificial intelligence has found fertile ground in software engineering, particularly in areas that benefit from automation, prediction, and contextual understanding. At its core, AI in software development leverages machine learning (ML), natural language processing (NLP), and deep learning to automate or augment various programming tasks.

Some of the most influential AI-driven tools in this space include:

GitHub Copilot: Powered by OpenAI's Codex model, GitHub Copilot provides real-time code completion suggestions directly within IDEs. It supports multiple programming languages and functions as an "AI pair programmer," reducing the time developers spend writing boilerplate or routine code (Yetiştiren et al., 2023). Copilot is now deeply integrated into popular IDEs such as Visual Studio Code, JetBrains IDEs, and Xcode, and offers features like chat-based assistance, code review suggestions, and even autonomous task execution through its new agent mode. Copilot can also generate commit messages, summarize pull requests, and create knowledge bases from documentation to aid team collaboration.

Amazon Q Developer (formerly CodeWhisperer): Amazon Q extends beyond code completion to offer features such as inline security scanning, code transformation, and multi-step task assistance. Unlike GitHub Copilot, which primarily focuses on code generation, Amazon Q aims to be a holistic developer assistant integrated across IDEs and AWS cloud environments (Yetiştiren et al., 2023).

AI-based Debugging Tools: Tools such as DeepCode (by Snyk), Facebook Infer, and Microsoft IntelliCode use ML models to detect code anomalies, suggest bug fixes, and recommend refactoring strategies. These tools can learn from large datasets of developer behaviour and codebases to offer context-sensitive guidance (Dwivedi, 2024).

These technologies are increasingly integrated into popular IDEs (e.g., Visual Studio, IntelliJ IDEA, VS Code) and version control platforms, allowing seamless adoption into existing developer workflows (Qian et al., 2024).

ERA	METHODLOGIES	KEY FEATURES	INTRODUCTION/ADOPTION PERIOD	REFERENCES
Traditional	Waterfall model	Linear, sequential approach to software development	1970s	(Royce, 1970)
Iterative	Agile, Scrum	Flexible, iterative approach emphasizing collaboration	1990s	(Manifesto, 2001)
Modern	DevOps, Continuous integration	Automated testing, continuous deployment practices	2010s	(Humble & Farley, 2010)
AI Driven	AI powered tools, ML, NLP	Automated coding, adaptive software maintenance	2020s	(Pearce et al., 2022; Qian et al., 2024)

Table 1 Evolution of software engineering practices by Alenezi, M., & Akour, M. (2025). AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. Applied Sciences, 15(3), 1344.(Alenezi & Akour, 2025)

2.2.2 Current Trends and Advancements

AI Tools in the Advancement of Software Systems:

Artificial Intelligence (AI) has become one of the most prominent and sought-after branches of computer science today. It empowers machines to behave intelligently and perform a wide range of tasks efficiently by prioritizing actions based on available data. As noted by Sahin (Şahin, 2022), AI serves as a catalyst that reduces human effort while continuously improving performance through iterative learning. The subject of AI continues to generate widespread interest in both academic and non-academic spheres. According to Bhandari et al. (2023) (Bhandari et al., 2023), AI techniques have a significant influence on the software development life cycle (SDLC), contributing meaningfully across its various phases. From design to maintenance, AI tools assist software developers by automating key stages of the SDLC. Figure 2 below illustrates the integration and impact of AI across different phases of software engineering.

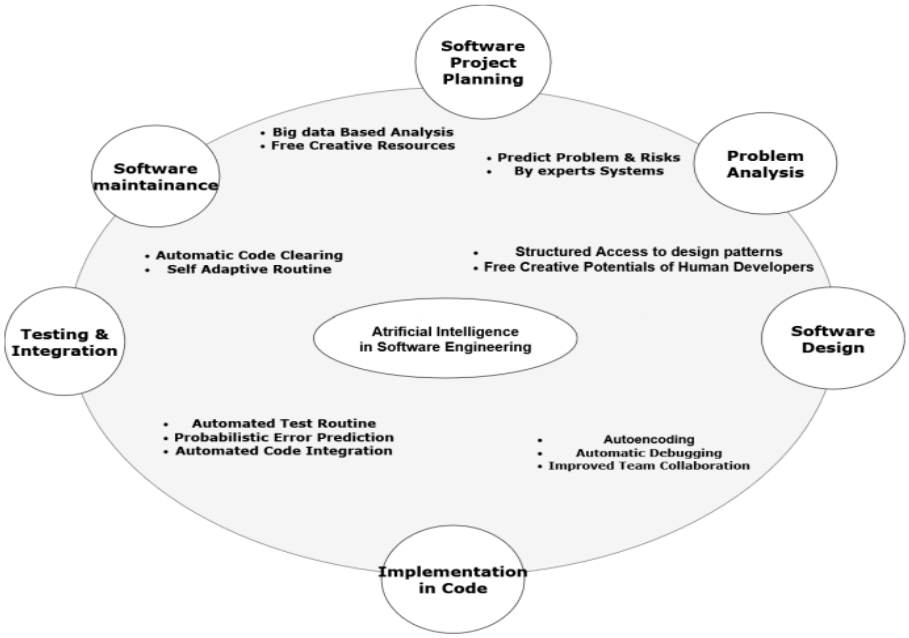

Figure 2 AI Tools in Software Development Life Cycle (Barenkamp et al., 2020)

The last three years have witnessed rapid evolution in the application of AI to software development. Some of the most prominent practices include:

AI-Assisted Pair Programming
GitHub Copilot has popularized the concept of AI-assisted pair programming, where the AI acts as a silent collaborator that continuously suggests lines or blocks of code as the human developer types. Recent advancements allow Copilot to take on more autonomous roles, such as handling multi-step tasks, code refactoring, and even implementing new features independently in agent mode. Empirical studies have shown that developers using Copilot complete tasks faster and maintain better mental flow (Szolderits, 2025).

AI for Code Review and Refactoring
AI is increasingly used to assist with code reviews by identifying potential anti-patterns, flagging technical debt, and offering recommendations for cleaner design. Copilot now provides automated code review suggestions and can generate summaries for pull requests, streamlining the review process and improving code quality. Autonomous agents can also perform code edits and refactoring across multiple files, reducing manual effort and human error (Ani et al., 2023).

Security and Vulnerability Detection
Amazon Q Developer integrates static analysis and vulnerability scanning directly into the code editing process. It flags common security issues (e.g., SQL injection, hardcoded secrets) and offers remediation suggestions. This integration reduces the reliance on post-build security audits and enables shift-left security (Araujo & Hayajneh, 2024).

Natural Language Interfaces for Code Queries
AI tools now support querying codebases using natural language, allowing developers to ask questions like “Where is the authentication middleware implemented?” or “Which method handles file uploads?” Amazon Q’s conversational interface makes it easier to navigate and understand large codebases, facilitating onboarding and knowledge transfer (Angara, 2024).

Code Summarization and Documentation
LLMs are capable of generating documentation summaries for functions, classes, or entire modules. Copilot can automatically create and manage knowledge bases from documentation, enabling developers to get context-aware answers to their questions and improving team collaboration (Yetiştiren et al., 2023).

AI for Testing and Bug Prediction

AI is transforming software testing and defect detection through two complementary approaches: predictive analytics and generative automation. Models trained on historical test and bug data identify failure-prone modules by analysing patterns in past defects, code complexity, and deployment history. This enables teams to prioritize QA efforts on high-risk areas, optimizing resource allocation (Bajaj & Samal, 2023).

Generative AI algorithms, such as LLMs and reinforcement learning models, automate test-case creation by synthesizing inputs from specifications, user stories, or system behaviour. These tools analyse codebases to uncover edge cases, generate comprehensive test suites with broad coverage, and even simulate user interactions (Bajaj & Samal, 2023).

In bug detection, AI systems scrutinize codebases, execution traces, and test results to flag anomalies like memory leaks, race conditions, or security vulnerabilities. Techniques such as pattern recognition (e.g., identifying SQL injection risks) and anomaly detection (e.g., deviations from normal API response times) enhance accuracy (Bajaj & Samal, 2023).

Advantages as recorded by Yatin Bajaj are as follows:

- Amplified test coverage: Generative models explore scenarios humans might overlook.
- Strategic efficiency: Predictive analytics focus QA efforts on critical areas, reducing redundant testing.
- Scalability: AI handles repetitive tasks, freeing developers for complex problem-solving.

Integration with DevOps and CI/CD
AI assistants are now being embedded into DevOps workflows, providing support for continuous integration, deployment, and monitoring. They can automate repetitive tasks, generate deployment scripts, and ensure best practices are followed throughout the software delivery pipeline (Rautiainen, 2023).

In summary, the integration of AI into software development is rapidly expanding, with tools like GitHub Copilot and Amazon Q Developer leading the way in automating routine tasks, enhancing code quality, and enabling new forms of collaboration and productivity across the software lifecycle

2.3 AI for Collaboration Enhancement

2.3.1 Existing Studies Highlighting AI in Collaborative Software Development Scenarios

The application of artificial intelligence in collaborative software engineering has rapidly matured, shifting from supporting individual developers to actively shaping team productivity, communication, and knowledge sharing in both co-located and distributed contexts.

Recent empirical research underscores the measurable impact of AI pair programmers like GitHub Copilot on collaborative open-source projects. A large-scale study by Song, Agarwal, and Wen (2024) analysed open-source repositories and found that Copilot use led to a 6.5% increase in project-level productivity and a 5.4% rise in developer participation. However, the study also observed a 41.6% increase in integration time, suggesting that while AI tools can boost output and engagement, they may introduce new coordination complexities that teams must manage (Song et al., 2024).

The study by Song, Agarwal, and Wen (2024) also illustrates that AI pair programming, as enabled by tools like GitHub Copilot, plays a transformative role in collaborative software development by offering real-time code suggestions that streamline the coding process and reduce development costs. Unlike traditional human pair programming, which involves one developer coding while the other reviews and strategizes, AI pair programmers automate much of the support role, enhancing speed and reducing the manual effort required for collaboration. The study acknowledges that while this can lead to faster code generation and broaden participation in open-source projects, it may also introduce new challenges such as increased integration overhead due to the influx of AI-assisted contributions. Nevertheless, the integration decisions made with the aid of AI during code generation can improve development flexibility and reduce downstream testing and merging conflicts, thereby reshaping how teams collaborate in distributed software ecosystems (Song et al., 2024).

Based on the comprehensive study titled “How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial” by Paradis et al. (Paradis et al., 2024), AI is demonstrably influencing software development by significantly accelerating task completion and shaping developer workflows. The study involved 96 full-time Google engineers and assessed the impact of three AI-enhanced tools, AI Code Completion, Smart Paste, and Natural Language to Code, on the time taken to complete a complex enterprise-grade development task.

The findings reveal that developers using AI tools completed tasks approximately 21% faster than those who did not, even when controlling for factors like experience, coding frequency, and task familiarity. These tools helped streamline code generation, automate context-aware pasting, and translate natural language prompts into executable code. Notably, developers who spent more time coding daily or had greater seniority benefited more from the AI enhancements, suggesting that familiarity and expertise amplify the productivity gains (Paradis et al., 2024).

Beyond raw speed, the study also underscores broader implications for AI integration in enterprise environments. It supports the hypothesis that AI enhances developer efficiency and promotes best practices by reducing friction in editing, refactoring, and documentation. However, it also cautions that these benefits are uneven: junior developers or those unfamiliar with AI tooling might experience lesser gains, and tasks requiring nuanced judgment may still demand human oversight.

This study (Paradis et al., 2024) provides robust empirical evidence that AI is not just a marginal aid but a transformative force in software development. Its influence spans from individual productivity to systemic workflow optimization, marking a shift toward AI-augmented engineering practices. It also raises important design considerations for future AI tools, such as personalization, trust calibration, and seamless integration into enterprise

ecosystems, thereby charting a path for their responsible and effective use in professional software engineering contexts.

The study by Sager and Wahab (Sager & Wahab, 2024) highlights how AI significantly enhances collaborative software development by improving productivity, streamlining workflows, and supporting team coordination. AI tools like GitHub Copilot and OpenAI Codex reduce the burden of repetitive coding tasks, enabling developers to focus more on high-level design and collaborative planning. These tools also enhance code quality and consistency by recommending best practices and offering real-time feedback, which supports asynchronous teamwork and reduces the need for extensive manual reviews. AI-driven debugging tools detect errors and suggest fixes, allowing teams to resolve issues more quickly and effectively, while AI-powered CI/CD integrations improve deployment transparency and alignment across distributed teams. Additionally, intelligent project management systems use predictive analytics to optimize resource allocation and balance workloads, fostering stronger collaboration and accountability. The study also notes improved developer satisfaction as a result of AI reducing routine workloads and facilitating smoother collaboration. However, it warns against over-reliance on AI, which could undermine developer engagement and introduce security risks if not properly overseen. Thus, the paper emphasizes the importance of balancing automation with human oversight to ensure software quality and maintain trust in collaborative environments (Sager & Wahab, 2024).

Based on the study "Collaborative Technologies in Global Engineering: New Competencies and Challenges" by Daniels, Cajander, Clear, and McDermott, AI in collaborative environments can be viewed through the lens of Collaborative Technologies (CTs) and the competencies required to mediate their use in globally distributed teams. While the study itself predates the widespread application of AI tools like GitHub Copilot or Amazon Q Developer, its findings remain highly relevant in understanding how AI integrates into such ecosystems (Daniels et al., 2015).

The research highlights that successful global collaboration is not just about using technology, but about mediating its use, a process now increasingly supported by AI. AI technologies enhance this mediation by streamlining communication, automating documentation, improving task visibility, and supporting synchronous and asynchronous coordination. For example, in scenarios where students or professionals must navigate differing time zones and cultural expectations, AI-driven summarization tools, natural language query interfaces, and adaptive scheduling assistants help reduce friction, improve mutual understanding, and support the establishment of common ground (Daniels et al., 2015).

Moreover, the notion of "Collaborative Technology Fit" (CTF), central to this study, maps directly onto current AI-enhanced collaborative platforms. AI can improve CTF by offering intelligent configuration suggestions, recommending tools or integrations based on usage patterns, and automatically resolving technology mismatches. However, the study also warns of the challenges in assuming technology is neutral or self-explanatory, an issue that persists with AI. Users often fail to reflect on how tools shape their interactions, leading to suboptimal appropriation or over-reliance without critical understanding (Daniels et al., 2015).

Therefore, incorporating AI into collaborative environments requires deliberate scaffolding in both educational and professional settings. This includes teaching developers how to use AI

tools not just technically, but socially and strategically, to facilitate inclusion, coordination, and shared knowledge-building. The study advocates for frameworks like Stahl's Collaborative Knowledge-Building Model and Constructive Controversy to support such reflection and dialogue, principles that are increasingly vital in navigating AI-mediated collaboration in software engineering (Daniels et al., 2015).

As per the study "Evolve from Infrastructure to Innovation with SAP on AWS" by Baruah, Ramadoss, and Vivekanandha (Baruah et al., 2024), Amazon Q Developer plays a pivotal role in enhancing collaborative software development, especially in distributed and enterprise environments. Amazon Q fosters multi-developer synchronization through features such as inline documentation, vulnerability scanning, and intelligent code transformation. Its seamless integration across popular IDEs and cloud-based development environments ensures that team members, regardless of their location, remain aligned on security practices, code standards, and architectural decisions. This is particularly valuable during onboarding, where new contributors can quickly adapt by leveraging AI-generated insights and contextual code suggestions. The tool also enables natural language input to generate scripts or automation workflows, thereby bridging skill gaps among team members and allowing non-specialist contributors to collaborate more effectively. Ultimately, Amazon Q supports not only efficient coding but also collaborative understanding, enabling teams to work smarter, faster, and more securely in complex enterprise settings (Baruah et al., 2024).

According to the study "Use Cases of Program Task using Tools based on Machine Learning and Deep Learning" by Hong and Hong (Hong & Hong, 2024), AI is significantly transforming collaborative software development, particularly through the integration of AI-powered debugging tools. Technologies like Microsoft IntelliCode and DeepCode utilize machine learning and deep learning models to automate code review, identify bugs, and recommend fixes, functions that traditionally required manual inspection by team members. These tools reduce the cognitive load on developers during collaborative debugging sessions and improve response times by learning from team behaviour and offering context-aware suggestions. In distributed development environments, this real-time, intelligent assistance ensures faster issue resolution, promotes code consistency, and enhances overall team productivity. By detecting vulnerabilities and automating parts of the review process, AI facilitates smoother collaboration and minimizes integration issues, ultimately contributing to higher-quality, more secure software outcomes (Hong & Hong, 2024).

Based on the existing study "Natural Language Query Engine for Relational Databases using Generative AI" by Steve Tueno (Tueno Fotso, 2024), AI significantly enhances collaborative software development by enabling conversational access to codebases and data through natural language interfaces. Tools like Amazon Q Developer and generative AI-powered query systems democratize access to technical project knowledge, allowing both developers and non-technical stakeholders to interact with databases and systems without needing SQL expertise. This greatly improves inclusivity and communication within distributed teams, where members with varying technical backgrounds can participate in decision-making, troubleshoot issues, or extract insights independently. By automating query translation, validating semantic and syntactic correctness, and generating clear natural language responses, AI bridges the gap between structured data and human understanding. This reduces dependency on gatekeepers (e.g., data engineers or DBAs), accelerates workflows, and ensures alignment across cross-functional teams, making AI a valuable enabler of seamless, collaborative software development (Tueno Fotso, 2024).

According to the study "A Comparative Study of AI Code Bots: Efficiency, Features, and Use Cases" by Mihir Mehta (Mehta, 2024), AI is enhancing collaborative software development by automating key documentation tasks that traditionally consume team time and contribute to communication gaps. Tools like Sourcegraph Cody and Tabnine Pro employ AI-driven summarization to automatically generate documentation, commit messages, and changelogs, ensuring that critical project knowledge is consistently captured and shared across the team. This capability reduces the cognitive burden on individual developers while improving shared understanding of evolving codebases. In collaborative settings, where multiple contributors work on interconnected modules, these AI-generated summaries foster collective code awareness and enable smoother onboarding, more effective reviews, and better continuity between contributors. As a result, AI not only accelerates development but also strengthens team alignment and long-term maintainability of software projects (Mehta, 2024).

According to Asif Hassan (Hassan, 2024) the below table shows the comparison between prominent AI tools used in software development.

Feature	**ChatGPT**	**GitHub Copilot**	**Stack Spot AI**	**Lingma**	**Tabnine**
Primary Focus	AI-powered conversational assistant	AI-powered code completion and suggestion	AI-powered code generation, templates, and automation	AI for automated workflows and code generation	AI-driven code autocompletion
Integration	Accessible via web or API	Integrated with Visual Studio, VS Code, JetBrains IDEs	DevOps platforms, project management systems	Visual Studio, VS Code	Available in VS Code, JetBrains IDEs, Sublime
Code Completion	Can generate code based on queries	Autocomplete based on context	Offers smart code snippets and templates	Autocompletes based on patterns	Suggests entire lines or blocks of code
Contextual Code Assistance	understand complex queries	Based on the surrounding code	With retrieval augmented generation (RAG)	Understands project specific patterns	using deep learning in various languages
Code Documentation	Can explain and generate documentation	Can provide comments and simple explanations	Offers in context documentation suggestions	Provides inline comments and documentation	Supports inline documentation lookup
Code Refactoring	Can suggest improvements when queried	Offers simple refactoring suggestions	Can refactor based on best	Provides refactoring hints	Basic refactoring suggestions

			practices retrieved		based on patterns
Strengths	Flexibility, multipurpose, can explain code in depth	Real-time code completion, integrated with GitHub	Code templates, DevOps integration, enterprise level automation	Workflow automation, DevOps and CI/CD focus	Versatile autocompletion across languages

Table 2 Various AI Coding Assistant Tools comparison on different feature (Hassan, 2024)

2.3.2 Identified Benefits and Drawbacks as per the existing studies

Benefits:

1. Improved Developer Productivity and Workflow Efficiency: AI tools significantly accelerate coding and review processes by automating repetitive and low-value tasks such as syntax validation, formatting, and boilerplate generation (Ahmed, 2025). This enables developers to focus on higher-order problem-solving and design tasks, enhancing team output and overall software quality.
2. Enhanced Consistency and Code Standardization Across Teams: AI-driven tools like Amazon Q Developer and GitHub Copilot enforce coding standards and reduce human errors. This leads to more uniform code contributions across team members, which is critical in large, distributed teams (Song et al., 2024) (Ahmed, 2025).
3. Faster Onboarding and Cross-Team Knowledge Sharing: With natural language interfaces (e.g., Amazon Q), even non-expert developers or new team members can query systems and understand unfamiliar codebases. This democratization of access supports faster onboarding and improved communication between technical and non-technical stakeholders (Tueno Fotso, 2024).
4. Collaborative Debugging and Issue Resolution: AI-powered debugging tools (e.g., IntelliCode, DeepCode) detect bugs, offer fixes, and learn from team behaviour to provide contextual recommendations. These tools facilitate real-time troubleshooting and reduce the mean time to resolution during collaborative debugging sessions (Hong & Hong, 2024).
5. Enhanced Code Documentation and Transparency: Tools such as Sourcegraph Cody and Tabnine Pro automatically generate documentation, commit messages, and changelogs. This eases cognitive load on developers and promotes collective code awareness, improving continuity and reducing knowledge silos (Mehta, 2024).
6. Security and DevOps Alignment: Integrated tools like Amazon Q Developer also embed security scanning and deployment support into collaborative workflows, ensuring alignment with DevOps principles and reducing vulnerabilities introduced through manual coding errors (Baruah et al., 2024) (Ahmed, 2025).
7. Continuous Learning and Developer Skill Growth: AI not only highlights issues but provides explanations for suggestions, helping junior developers learn from the process. This creates an ongoing learning loop within Agile teams, fostering growth and reducing the mentoring burden on senior staff (Ahmed, 2025).

Drawbacks:

1. Automation Bias and Over-Reliance on AI Suggestions: Developers, especially less experienced ones, may over trust AI outputs, accepting flawed or insecure code

without adequate scrutiny. This over-reliance can compromise the overall codebase integrity and lead to potential security issues (Pearce et al., 2022) (Song et al., 2024).

2. Interpretability and Review Complexity: AI-generated code can sometimes be difficult for team members to interpret or validate, complicating peer reviews. This disrupts the collaborative review process and can reduce accountability or introduce bugs during integration (Ahmed, 2025) (Martínez-Fernández et al., 2017).
3. Resistance to Adoption and Change: Some developers perceive AI as a threat to job security or autonomy in decision-making. This can lead to resistance, fragmented usage, or lack of trust in AI suggestions, ultimately undermining collaboration (Ahmed, 2025).
4. Tool Integration Challenges: AI tools may not be seamlessly compatible with existing CI/CD pipelines, legacy codebases, or project management systems. This leads to fragmented workflows and inefficiencies, especially in established teams with mature processes (Ahmed, 2025; Baruah et al., 2024)
5. Accuracy and Contextual Limitations: AI models trained on generic datasets often struggle with context-specific logic, leading to false positives or overlooked issues. Human oversight remains essential to avoid critical failures in collaborative software contexts (Ahmed, 2025).
6. Legal and Ethical Uncertainty: AI-generated code, especially from models trained on public data, raises licensing, copyright, and compliance concerns in shared development environments. Teams may unknowingly inherit legal liabilities through copied or improperly attributed code (Song et al., 2024).

In summary, while AI-powered tools are demonstrably reshaping collaborative software development, improving productivity, knowledge sharing, and security, they also introduce new complexities in coordination, review, and compliance. Ongoing research and careful tool integration are essential to fully realize the benefits while mitigating the risks.

2.4 Research Gap

Although the integration of AI into software engineering has made significant strides, its application in collaborative settings remains insufficiently explored. Most of the existing literature considered in this research overwhelmingly focuses on individual productivity metrics, such as coding speed or task completion time, without adequately evaluating the impact of AI tools on team-centric variables such as communication, decision-making, shared understanding, code comprehension across developers, and the sustainability of AI-generated code in shared environments. This skewed focus has left major knowledge gaps around the adoption and practical efficacy of AI tools in multi-developer environments. There is a conspicuous lack of holistic evaluation on how AI tools impact such collaboration-specific dimensions.

1. **Underrepresentation of Team-Level Collaboration Metrics**

Most evaluations of AI tools (e.g., GitHub Copilot, Amazon Q) prioritize individual productivity. For example, a 2023 Microsoft study found that Copilot users wrote code 55% faster but noted no significant analysis of team outcomes like reduced merge conflicts or improved code review efficiency. Similarly, AWS's 2024 case study on Amazon Q highlighted a 30% reduction in individual task time but lacked metrics on cross-developer alignment or shared context.

Recent work by Treude et al. (2025) systematically reviewed relevant AI-in-software-engineering studies and found that only 12% measured team-level outcomes (e.g., communication patterns, decision-making efficiency). This gap is critical, as collaborative development relies on synchronized workflows and collective code ownership, factors AI tools may inadvertently disrupt through fragmented code generation styles (Treude & Storey, 2025).

2. **Lack of Longitudinal Evidence**

Industry reports, such as GitLab's 2024 Global Developer Report, reveal that 68% of teams using AI tools experience initial productivity spikes but struggle with technical debt accumulation over 6–12 months (GitLab, 2024). For instance, a longitudinal study of GitHub projects by Hilton et al. (2024) found that AI-generated code contributed to a 22% increase in code churn (post-merge revisions) compared to human-authored code, suggesting maintainability risks in collaborative settings (Hilton et al., 2016).

3. **Limited Understanding of AI's Role in Communication and Knowledge Transfer**

While tools like Amazon Q Developer provide inline documentation, their impact on team knowledge sharing is poorly understood. A 2021 ethnographic study of distributed teams at IBM (Piorkowski et al., 2021) found that AI-generated summaries reduced onboarding time by 40% but also led to over-reliance on automation, with developers skipping critical design discussions. Conversely, teams using AI for real-time code explanations (e.g., Sourcegraph Cody) reported 35% fewer misunderstandings during code reviews (Piorkowski et al., 2021).

4. **Security, Ethical, and Legal Ambiguity in Collaborative Settings**

Studies reveal systemic risks in team environments:

- Fu et al. (2025) found that 41% of AI-generated code snippets in team projects contained licensing conflicts from training data (Fu et al., 2025).
- A 2024 OpenSSF report showed that AI-assisted teams introduced 2.3× more vulnerabilities than manual coding groups, as developers often bypassed peer reviews for AI-suggested code (Gupta, 2024).

Research Direction Justification

This dissertation directly responds to the identified gaps by conducting a systematic mapping study focused on the collaborative dimensions of AI tool usage in software development. Rather than isolating tools or features, this study takes a thematic and role-aware approach to assess how AI technologies shape shared workflows, coordination mechanisms, and team-level productivity.

The research will provide:

- A categorized overview of the AI tools currently adopted in collaborative contexts.
- An evaluation of their functional strengths and integration barriers.
- Insights into how these tools enhance or inhibit collaboration, particularly in distributed teams.
- A foundation for future empirical validation, tool design enhancements, and organizational policies.

By identifying gaps and proposing areas for methodological and technical development, this study contributes not only to academic literature but also to the practical evolution of AI-supported collaborative software engineering.

CHAPTER 3: RESEARCH METHODLOGY

3.1 Research Approach

3.1.1 Justification for Choosing Systematic Mapping Study

This research adopts a **Systematic Mapping Study (SMS)** approach to comprehensively identify, categorize, and evaluate existing literature on the application of AI-driven tools within collaborative software development. A systematic mapping study was selected due to its suitability for structuring broad research areas, identifying gaps, and highlighting potential research directions without conducting an exhaustive synthesis typical of systematic reviews. Given the nascent yet rapidly evolving nature of AI integration in collaborative software development, a systematic mapping study provides a clear overview of current knowledge, practices, and gaps, making it particularly valuable in emerging fields.

As we can see in the figure 3 (Petersen et al., 2008), the important process steps of our systematic mapping study are Definition of research question, that is the primary goal of this study is to provide an overview of a research area and identify the quantity and type of research and results available in this area; Conducting the search for relevant papers, Papers were identified using search strings on scientific databases; Screening of papers, using inclusion and exclusion criteria to exclude studies that were not relevant for the research questions; Keywording of abstracts, which is a way to reduce the time needed to develop the classification scheme hence making sure the method considers existing studies; Data extraction and mapping, relevant articles were sorted into the scheme once the classification scheme was in place; with each process steps having an outcome, systematic map being the final outcome (Petersen et al., 2008; Sofian et al., 2022).

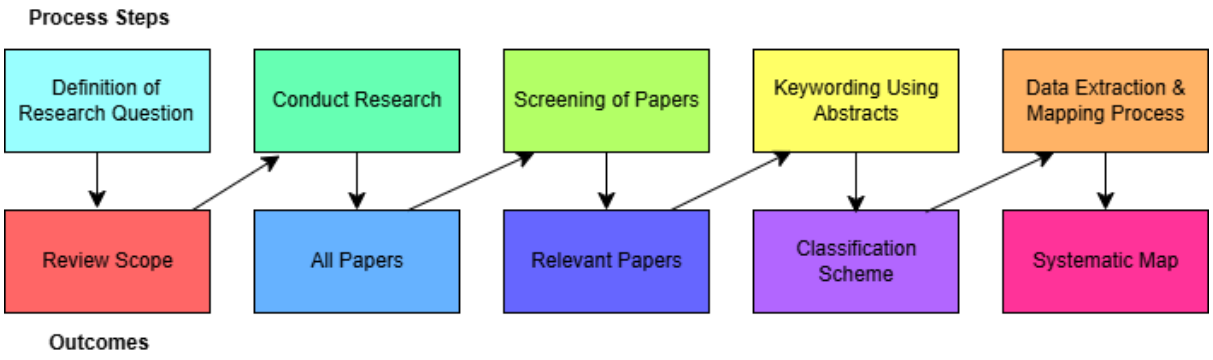

Figure 3 The systematic mapping process created using Draw.io

Specifically, the SMS approach allows for:

- **Broad Coverage and Categorization**: The scope of AI in collaborative software engineering is extensive, encompassing tools, techniques, methodologies, and multiple software lifecycle phases. A mapping study effectively captures this breadth, enabling classification based on well-defined criteria.

- **Identification of Research Trends and Gaps**: The structured categorization provided by systematic mapping studies helps in clearly identifying areas that are well-researched, as well as areas that require further empirical studies or deeper investigation.
- **Foundation for Further Research**: By highlighting existing knowledge gaps, the systematic mapping study acts as a valuable starting point for subsequent systematic reviews, controlled experiments, or empirical research aimed at exploring deeper insights or hypotheses.

3.1.2 Difference Between Mapping Study and Literature Review/Systematic Review

Systematic mapping studies, systematic reviews, and traditional literature reviews serve distinct but complementary purposes within research. Key differences include (Petersen et al., 2008):

Systematic Mapping Study (SMS):

- Objective: To provide a structured and broad overview of a specific research area, identifying existing themes, patterns, and research gaps.
- Scope: Typically, broader and less in-depth in terms of individual study analyses. It involves categorizing a large volume of literature according to predefined classification schemes.
- Outcome: Delivers a visual or tabular mapping of research activity and topics covered, often without detailed synthesis or evaluation of results from individual studies.

Systematic Review (SR):

- Objective: To rigorously collect, critically evaluate, and synthesize empirical evidence related to a narrowly defined research question.
- Scope: More focused and exhaustive, usually assessing the quality and outcomes of included studies in detail to draw specific conclusions or establish robust evidence-based recommendations.
- Outcome: Results in a detailed synthesis, typically quantitative (meta-analysis) or qualitative, that answers clearly defined research questions.

Traditional Literature Review:

- Objective: To broadly summarize and interpret existing knowledge without a rigid methodology or detailed inclusion/exclusion criteria.
- Scope: Typically, less systematic; studies selected based on their perceived relevance by the author without structured searches.
- Outcome: Provides a narrative synthesis, highlighting general trends and theories rather than systematic categorization or critical evaluation.

In the context of this dissertation, an SMS approach was chosen to systematically map out existing literature related to the impact of AI tools on collaborative software development, thus laying the groundwork for more targeted future research and systematic reviews.

3.2 Research Questions

3.2.1 Main Research Question

The primary research question **(RQ1)** guiding this systematic mapping study is:

"What is the impact of AI-driven tools and techniques on collaborative software development practices?"

Rationale: This overarching question addresses the core aim of the research, examining how AI integration influences team efficiency, communication effectiveness, and software quality within collaborative software engineering environments.

3.2.2 Sub-Questions Guiding the Mapping Study

To further structure and direct the systematic mapping study, the following sub-questions are formulated:

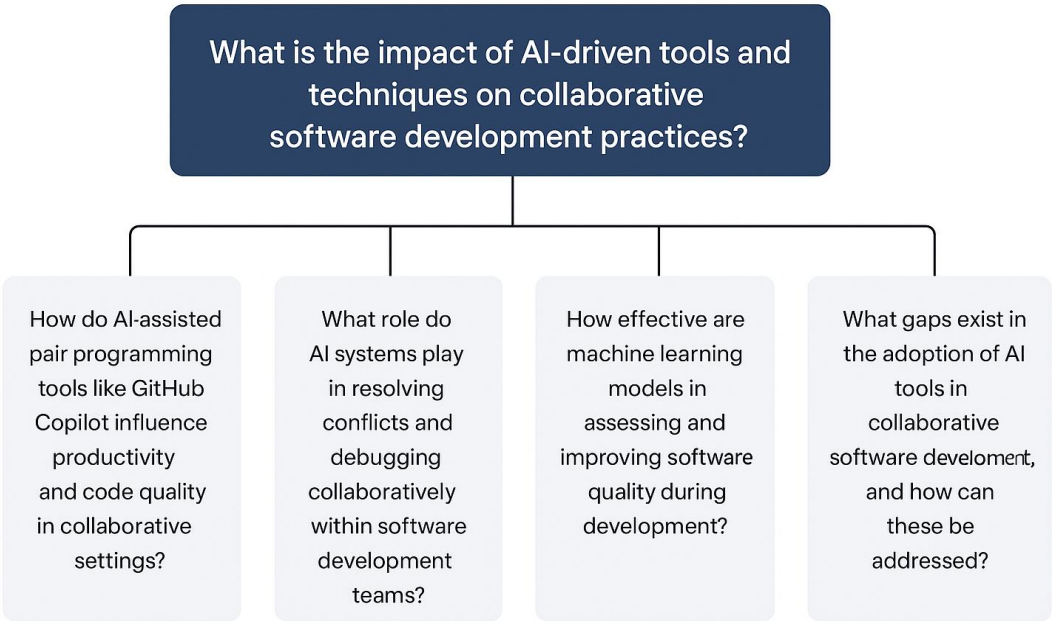

Figure 4 Research question and Sub questions of this mapping study

1. **(RQ1.1) How do AI-assisted pair programming tools like GitHub Copilot and Amazon Q influence productivity and code quality in collaborative settings?**
   Rationale: This question investigates the extent to which AI pair programming tools enhance coding efficiency, reduce errors, and improve software quality, focusing particularly on collaborative and distributed team contexts.

2. **(RQ1.2) What role do AI systems play in resolving conflicts and debugging collaboratively within software development teams?**
   Rationale: This explores how AI-driven debugging tools contribute to conflict resolution and problem-solving in collaborative environments, specifically focusing on their ability to streamline collaborative debugging and reduce integration complexities.

3. **(RQ1.3) How effective are machine learning models in assessing and improving software quality during development?**
   Rationale: This question addresses the capabilities of AI-driven machine learning models in evaluating software quality metrics, detecting vulnerabilities, predicting

defects, and suggesting quality improvements throughout the software development lifecycle.

4. **(RQ1.4) What gaps exist in the adoption of AI tools in collaborative software development, and how can these be addressed?**
   Rationale: This aims to systematically identify under-explored areas, barriers, and limitations of current AI-driven tools in collaborative settings. Additionally, it seeks to provide actionable insights and recommendations to overcome these barriers, facilitating better integration and utilization of AI tools.

These sub-questions are designed to comprehensively cover critical aspects of the main research question, enabling an effective mapping and categorization of the current state of knowledge in AI-supported collaborative software engineering.

3.3 Search Strategy

3.3.1 Definition of Search Terms and Keywords and Query Strings Used

To systematically conduct the search and ensure broad yet focused coverage of the literature, the search strategy was developed using the **Population, Intervention, Comparison, and Outcome (PICO)** framework as suggested by the study "Systematic literature reviews in software engineering- A systematic literature review" (Kitchenham et al., 2009);

The PICO (Population, Intervention, Comparison, and Outcome) framework is a structured approach commonly employed in systematic mapping studies and systematic literature reviews to formulate clear, targeted research questions and effective search strategies. It helps ensure that the study thoroughly captures relevant literature, providing clarity and precision in research (Sofian et al., 2022).

Here's how each component applies specifically to your research:

- Population (P)

Definition: The target group or subject of the research.

In this study:

- The target population is software development teams involved in collaborative software engineering.
- This includes both co-located and geographically distributed teams engaging in collective software development tasks like coding, debugging, testing, or deployment.

- Intervention (I)

Definition: The primary technique, tool, method, or strategy being evaluated or analysed.

In this study:

- The intervention is AI-driven tools and techniques.
- This broadly includes Machine Learning (ML), Deep Learning (DL), Natural Language Processing (NLP), predictive models, automation algorithms, and various other AI methodologies.
- Example tools include specific platforms such as GitHub Copilot, Amazon Q Developer and ChatGPT, known for assisting developers in collaborative software tasks.

- Comparison (C)

Definition: The alternative methods or traditional practices against which the intervention is compared.

In this study:

- The comparison involves traditional or non-AI-driven software development practices.
- Examples include conventional pair programming without AI, manual debugging processes, traditional software testing, manual code reviews, and conventional software deployment practices.

- Outcome (O)

Definition: The results, impacts, or measures assessed in the study.

In this study: The outcomes are the impacts on collaborative software development, specifically concerning:

- Team Efficiency (e.g., how quickly tasks are completed, resource utilization)
- Communication Effectiveness (e.g., clarity of interaction, reduction in misunderstandings)
- Software Quality (e.g., reduction in defects, improved code maintainability)
- Conflict Resolution and Debugging (e.g., speed and efficiency of resolving coding conflicts or debugging errors collaboratively).

Based on these elements, the following comprehensive search string was constructed to maintain consistency across multiple databases:

("Artificial intelligence" OR "AI" OR "Machine learning" OR "Deep learning" OR "Natural language processing" OR "Data mining" OR "Predictive models" OR "AI automation" OR " AI integration") ***AND*** *("Collaborative software development" OR "Collaborative computing" OR " Software Teams" OR "Software engineering" OR "Pair programming" OR "Debugging" OR "Software testing" OR "Continuous integration" OR "Continuous deployment" OR "Team productivity" OR "Code quality" OR "Software quality" OR "Security and risks" OR "Systematic review" OR "Mapping studies" OR "Literature review")* ***AND*** *("GitHub Copilot" OR "Amazon CodeWhisperer" OR "Amazon Q Developer" OR "ChatGPT" OR "AI assistant" OR "AI tools" OR "DevOps")*

Here, the first (OR) segment targets studies involving various AI methods and related terms. It broadly captures papers mentioning AI explicitly or implicitly through techniques like machine learning, NLP, automation and integration. The second portion focuses explicitly on collaborative software development practices and relevant activities, ensuring that retrieved studies relate directly to software engineering tasks performed collaboratively. And the final segment explicitly targets studies involving prominent AI-driven tools specifically known to support collaborative software development. This ensures relevance and specificity, narrowing the search to research explicitly addressing AI tools.

Purpose of using PICO:

By employing the PICO framework, this systematic mapping study benefits in several keyways (Hosseini et al., 2024):

- **Clarity and Focus**: Clearly defining each element helps ensure the research stays targeted on precisely what I intend to investigate.
- **Consistency**: It promotes consistency across searches in various academic databases, facilitating reproducibility and transparency.
- **Comprehensive yet specific**: It balances breadth (capturing a wide range of relevant literature) with depth (focusing specifically on this research topic).
- **Effective Results**: Improves the likelihood of retrieving high-quality, relevant, and impactful academic articles directly addressing the research questions.

Thus, using PICO in the search strategy ensures a methodologically rigorous approach, significantly enhancing the quality and precision of this SMS.

3.3.2 Databases Selected

To ensure comprehensive coverage and retrieval of high-quality, peer-reviewed literature relevant to the study, the following widely recognized and respected academic electronic databases were selected:

- **IEEE Xplore**
- **ACM Digital Library**
- **Scopus**
- **ScienceDirect**
- **SpringerLink**

These databases are considered leading sources of high-impact peer-reviewed articles and conference proceedings within the field of software engineering and computer science.

3.3.3 Inclusion and Exclusion Criteria

To systematically select relevant studies, clear inclusion and exclusion criteria were established as follows:

Inclusion Criteria	
IC1	Articles that are peer-reviewed
IC2	Articles explicitly addressing collaborative software engineering scenarios with AI intervention
IC3	Most recent studies to capture the latest advancements in AI-driven software development practices
IC4	Studies that have been cited more than five times, indicating the relevance and acceptance within the academic community
Exclusion Criteria	
EC1	Articles not meeting the inclusion criteria
EC2	Studies available only as abstracts or presentations, lacking sufficient detail
EC3	Studies published in languages other than English
EC4	Studies lacking validation or empirical support for the proposed techniques or interventions
EC5	Articles providing unclear results, opinion pieces, or editorial commentary rather than empirical data or validated findings

Table 3 Inclusion and Exclusion Criteria of this SMS

These criteria ensured rigorous screening, helping focus the review on the most relevant, high-quality studies directly addressing the research questions posed by this systematic mapping study. This structured approach for the search strategy helps maintain transparency, reproducibility, and rigor in your systematic mapping study.

3.4 Selection and Screening Process [PRISMA Diagram]

3.4.1 Methodology for Screening and Selecting Studies

To ensure the selection of high-quality, relevant literature for this systematic mapping study, a structured two-phase screening process was utilized. This systematic approach follows guidelines proposed by (Petersen et al., 2008; Sofian et al., 2022) and the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) updated guidelines by (Page et al., 2021):

Phase 1: Initial Screening (Title and Abstract)

- Initially, retrieved papers from the selected databases (IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect, and SpringerLink) were screened based on their titles and abstracts.
- During this phase, studies were rapidly evaluated against the predefined inclusion and exclusion criteria.
- Papers clearly not relevant or not meeting inclusion criteria, such as articles lacking AI interventions in collaborative software settings or those not peer-reviewed, were excluded at this stage.

Phase 2: Full-Text Screening

- Papers shortlisted from the initial screening underwent a detailed full-text review.
- Each paper was carefully analysed to verify alignment with inclusion criteria such as explicit use of AI-driven collaborative software engineering practices, clear empirical validation of results, recent publication, and citations greater than five.
- Studies failing to meet these detailed inclusion standards or those presenting unclear results, opinion-based arguments, or insufficient empirical validation were excluded.

Summary of Screening Methodology Steps:

1. Database Search: Initial search and retrieval from databases.
2. Duplicates Removal: Identifying and removing duplicate studies.
3. Title and Abstract Screening: Rapid assessment for preliminary relevance.
4. Full-Text Screening: Detailed review against rigorous inclusion/exclusion criteria.
5. Final Selection: Compilation of a refined list of studies included for data extraction and analysis.

3.4.2 Tool Used (PRISMA Flow Diagram for Visual Representation)

To visually illustrate and document the study selection and screening process clearly, the PRISMA flow diagram was utilized. PRISMA is widely recognized in systematic studies as it enhances transparency and facilitates clear communication of the research methodology.

PRISMA Diagram Components Explained:

- Identification: Shows the total number of studies initially identified from database searches and other sources.

- Screening: Presents the number of studies screened at the title and abstract level, along with duplicate removal.
- Eligibility: Indicates how many studies were assessed at the full-text level and provides reasons for exclusion at this detailed screening stage.
- Included: Shows the final count of studies included for data extraction and further systematic analysis.

The adoption of the PRISMA diagram significantly contributes to the clarity, reproducibility, and transparency of the research process. It enables peers, reviewers, and readers to clearly understand the study selection criteria and workflow, thus enhancing the methodological rigor of this systematic mapping study. Figure 5 shows the PRISMA flowchart for this systematic mapping study, with 'n' representing the number of records/papers/studies involved and 'N' being the final references included in this study.

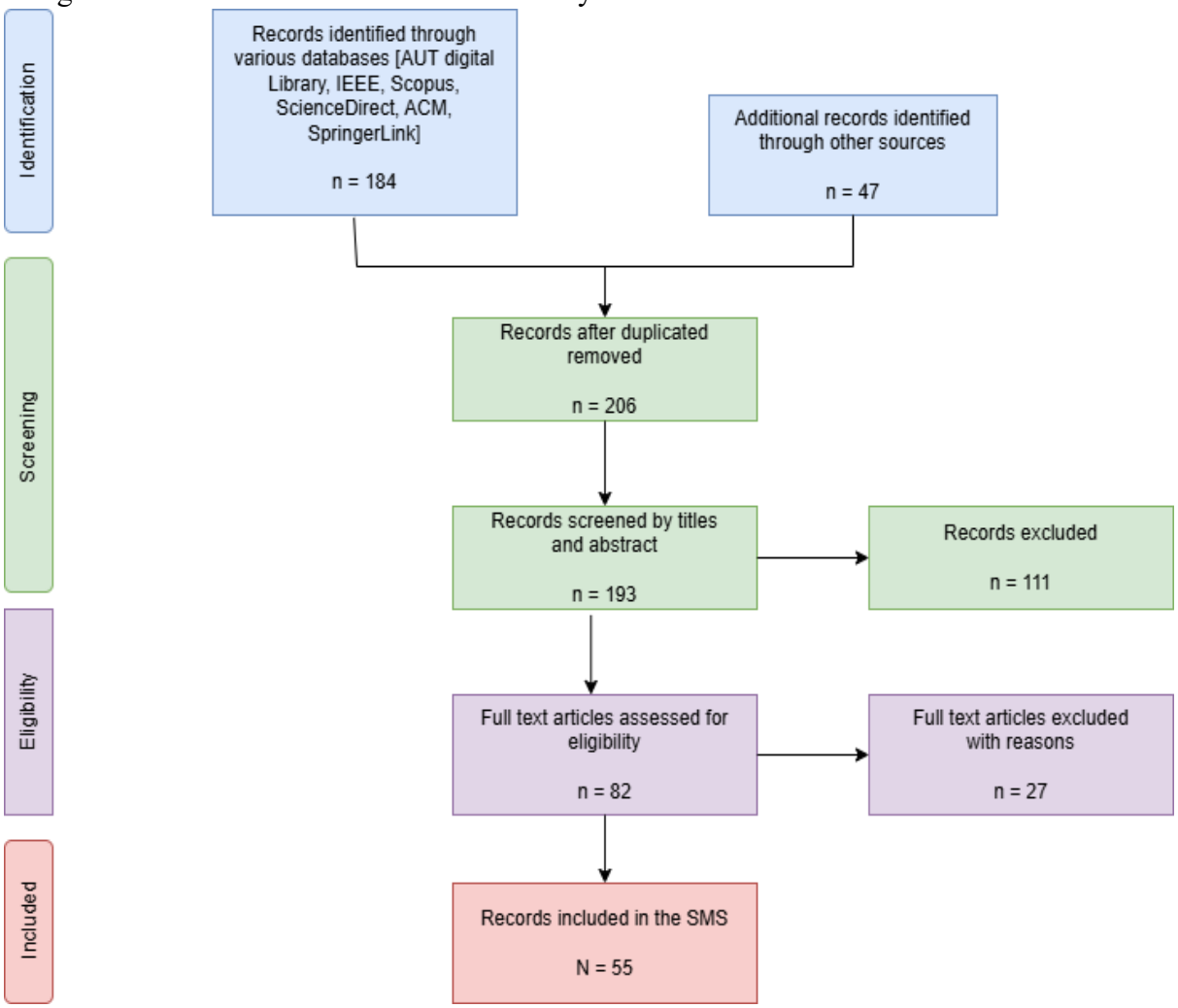

Figure 5 PRISMA flowchart for this mapping study created using Draw.io

3.5 Classification Scheme

3.5.1 How Studies Are Categorized and Classified

In alignment with the systematic mapping approach outlined by Petersen et al. (Petersen et al., 2008), this study adopted a facet-based classification scheme developed through an

iterative process of keywording abstracts and thematic synthesis. This method provides a structured yet adaptable framework to organize diverse studies across the collaborative software engineering domain where AI is applied.

The classification process was conducted as follows:

1. **Keywording of Abstracts**
   Each paper's abstract and occasionally the introduction or conclusion when abstracts lacked detail was reviewed and analysed to extract meaningful terms and concepts. These keywords represented the main contribution, mapping study context, AI techniques employed and target collaborative software development activities.

2. **Clustering and Categorization**
   Extracted keywords were grouped into thematic clusters, forming the basis for high-level categories. This clustering process ensured that the classification scheme accurately reflected the empirical literature and aligned with the core objectives of the mapping study.

3. **Facet-Based Classification**
   To allow for a multidimensional analysis, the studies were categorized along the following three primary facets:

- **AI Technique Used**: This facet captured the underlying AI approach used in the study:
  - Machine Learning (ML) – including supervised, unsupervised, and reinforcement learning
  - Deep Learning (DL) – convolutional/recurrent neural networks and transformers
  - Natural Language Processing (NLP) – code summarization, language modelling, conversational AI
  - Hybrid Approaches – combinations of multiple AI methods
- **Software Development Phase Targeted**: Identifying the stage of the SDLC addressed:
  - Requirements Engineering
  - Software Design
  - Implementation & Coding
  - Testing & Quality Assurance
  - Deployment & Maintenance
- **Collaborative Activity Supported**: Capturing the collaborative aspect being facilitated:
  - Pair Programming
  - Code Review and Refactoring
  - Debugging and Fault Resolution
  - CI/CD Automation
  - Documentation and Knowledge Sharing

4. **Research Type Facet** (Wieringa et al., 2006)
   This facet classifies studies by the type of research methodology or academic contribution, enabling analysis of both empirical rigor and innovation:

- Validation Research: Controlled lab-based evaluation of proposed techniques
- Evaluation Research: Field evaluation in industrial or real-world settings
- Solution Proposals: Novel approaches or enhancements with conceptual justification
- Experience Reports: Practitioner insights and case studies based on real deployment
- Philosophical Papers: Theoretical or conceptual frameworks, taxonomies, or models
- Opinion Papers: Viewpoints or expert commentary without empirical grounding

Rationale and Significance of This Classification

The multidimensional classification offers a comprehensive map of the research landscape, allowing for Identification of dominant research directions (e.g., heavy focus on implementation or NLP-based tools), Detection of underexplored intersections, such as under-representation of AI-supported code review in the deployment phase, Comparison of research maturity, distinguishing between proposal-heavy and evaluation-rich subareas and Mapping of real-world vs. theoretical research balance, guiding practitioners and researchers alike.

This structured approach enables not only a granular review of the literature but also provides actionable insights into areas needing further empirical investigation, practical validation, or conceptual refinement.

3.6 Data Extraction and Mapping of Studies

3.6.1 Data Extraction Templates or Forms

In line with the goals of a systematic mapping study focused on AI applications in collaborative software development systems, a structured data extraction template was designed to capture the collaborative context, intervention type, and outcomes of each selected study.

This form was used consistently throughout the review process to record and organize essential metadata about each paper and ensure traceability, comparability, and transparency.

Data Extraction Form – Key Fields and Descriptions:

Field	Description
Study ID	Unique reference number for each study
Title and Authors	Full title and author list for citation and traceability
Year of Publication	Indicates temporal trends in research focus
AI Technique or Tool Used	Specifies the AI system applied (e.g., GitHub Copilot, Amazon Q, ChatGPT)
Collaborative Development Practice	Specific practice within collaborative software development addressed by the study. The primary practices identified include Pair Programming, Model Testing, code review, Refactoring, effort Estimation
Supported Collaborative Activity	Team-oriented actions enabled by AI tools, such as task coordination, version control collaboration, or live code suggestions

Research Type (Wieringa et al., 2006)	Type of research approach: Empirical, Evaluation, Proposal, Solution proposal, Philosophical, Case study, Comparative, Security assessment or Application study
Contribution Type	Nature of the contribution (e.g., AI-assisted development framework, automation tool, empirical model, process enhancement)
Collaboration-Oriented Outcome Metrics	Evaluation metrics used, such as collaboration efficiency, defect reduction, shared code quality, or improved team communication
Summary of Findings	Concise synthesis of the study's main outcomes and relevance to collaborative AI-assisted practices

Table 4 Data extraction form with key fields and descriptions (Petersen et al., 2008; Wieringa et al., 2006)

Each field was filled based on a thorough review of the selected article, with a short justification note for the assigned values. When information was insufficient in the abstract, the introduction, methodology, and conclusion sections were examined to ensure accuracy.

3.6.2 Systematic Map Visualization: Scatter Plot

To visualize the distribution and frequency of AI applications in collaborative development contexts, a scatter plot is created. The visual mapping supports a comparative understanding of research activity across multiple dimensions.

The table 4 maps out the relationships between AI tools and collaborative software development practices as derived from the selected studies in the mapping study.

AI Tool	Collaborative Practice	Study Title	Research Type
GitHub Copilot	Pair Programming	Friedmann (2024)	Evaluation
GitHub Copilot	Refactoring	Szolderits (2025)	Evaluation
GitHub Copilot	Code Review	Fu et al. (2025)	Empirical
GitHub Copilot	Model Testing	Song et al. (2024)	Empirical
GitHub Copilot	Effort Estimation	Mehta (2024)	Comparative
Amazon Q	Model Testing	AWS Blog (2024)	Case study
Amazon Q	Effort Estimation	Araujo & Hayajneh (2024)	Security Assessment
Amazon Q	Code Review	Ahmed (2025)	Solution Proposal
ChatGPT	Pair Programming	Yetiştiren et al. (2023)	Empirical
ChatGPT	Model Testing	Tueno Fotso (2024)	Application Study
ChatGPT	Refactoring	Yetiştiren et al. (2023)	Empirical

Table 5 Study data table to show the mapping

Key takeaways:

- GitHub Copilot appears across all five collaborative practices, indicating it is the most widely studied tool in this domain. For example:
    - Friedmann (2024) explores its use in pair programming.
    - Szolderits (2025) focuses on refactoring.
    - Mehta (2024) assesses its performance in effort estimation using a comparative study.
- Amazon Q is studied in three areas, model testing, effort estimation, and code review, demonstrating growing interest in its capabilities beyond just Q&A:
    - Araujo & Hayajneh (2024) conducted a security assessment on Amazon Q's generated code.
    - Ahmed (2025) proposed it as a solution for automated code reviews.
- ChatGPT is involved in pair programming, model testing, and refactoring, suggesting that while it is versatile, it is less frequently studied in structured CI/CD or effort estimation workflows.
    - Yetiştiren et al. (2023) appears twice, showing its application in both pair programming and refactoring.
- Research Types vary across tools and practices:
    - GitHub Copilot is most commonly assessed via evaluation and empirical studies.
    - Amazon Q's usage spans case studies, security analysis, and solution proposals.
    - ChatGPT studies are mostly empirical or application-focused, highlighting its experimental and exploratory nature in development teams.

This table is constructed in such a way that the selected studies each cover at least one of the defined collaborative development practices with respect to the AI tools assessed in this research. This approach ensures balanced representation across both tools and practices, enabling a more comprehensive analysis of how different AI assistants are currently applied within team-oriented software engineering tasks. This table not only clarifies which tools are being used for what purposes but also reveals the depth and type of academic inquiry associated with each pairing, insightful for identifying both well-covered and underexplored intersections in this systematic mapping study.

Design of the Scatter Plot

To visualize the results of the classification and mapping process, a categorical scatter plot was created to represent the distribution of individual studies across various combinations of AI coding assistant tools and collaborative software development practices. Each data point represents a unique study and is color-coded according to its research type.

The scatter plot is designed with the following structure:

- X-Axis - AI Coding Assistant Tools:
  The horizontal axis displays commonly used AI development tools such as GitHub Copilot, Amazon Q, and ChatGPT. These tools are integrated into modern

development workflows to support tasks like real-time code assistance, documentation, and intelligent debugging.

- Y-Axis - Collaborative Development Practices:
  The vertical axis categorizes essential collaborative practices in software development. Based on the literature reviewed in this study, the key practices include: Pair Programming, Model Testing, Code Review, Refactoring, and Effort Estimation.
- Colour Coding - Research Type:
  Each point on the scatter plot is assigned a unique colour that corresponds to the research methodology used in the associated study, such as Empirical, Evaluation, Case Study, or Solution Proposal. A legend is provided to help interpret the colour mappings.
- Uniform Marker Size:
  Unlike traditional bubble plots, this scatter plot uses uniform-sized markers, as each point represents a single study rather than aggregated counts.

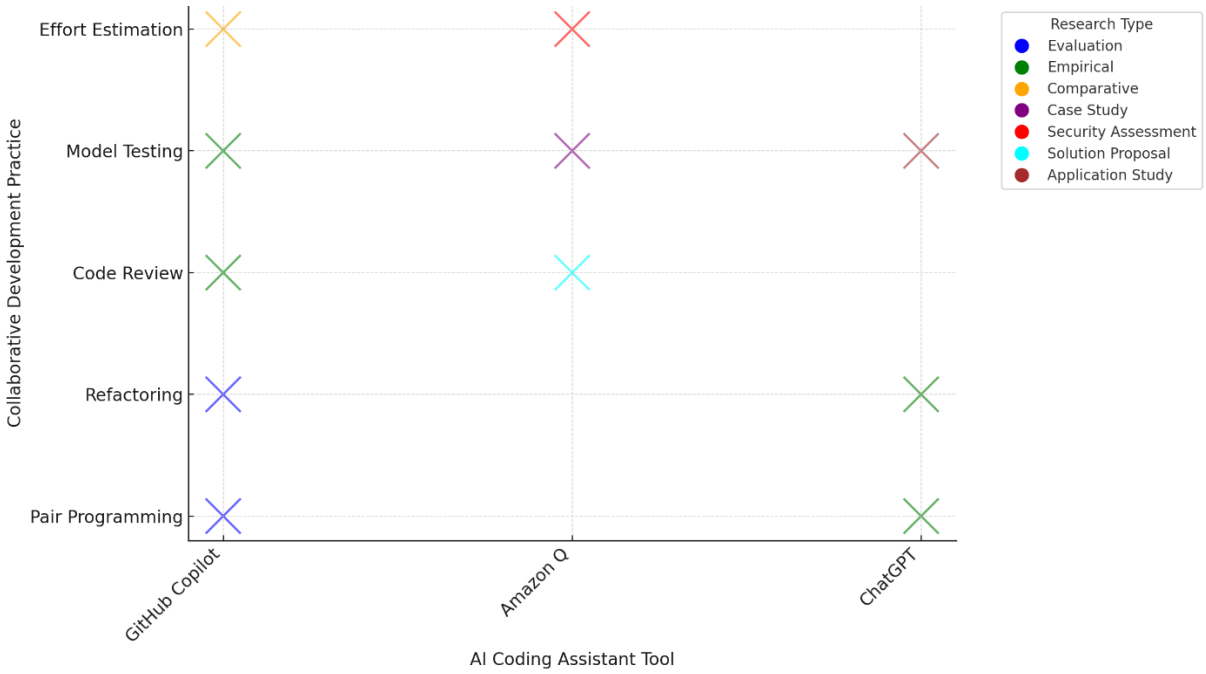

Figure 6 Scatter plot AI Tools v/s Collaborative practices

Purpose of the Visualization

The scatter plot supports multiple analytical objectives within the context of this systematic mapping study:

1. Highlighting Tool-Practice Coverage:
   The plot shows how specific AI tools have been studied in relation to various

collaborative development practices. For example, the presence of points linking GitHub Copilot to pair programming and refactoring highlights active research areas.

2. Identifying Research Gaps:
   The absence of points in certain tool-practice intersections (e.g., ChatGPT and Effort Estimation) indicates underexplored areas, guiding future research directions.
3. Visualizing Research Diversity:
   By incorporating research type as a color-coded dimension, the plot enables readers to understand not only what combinations have been studied, but also how, whether through evaluation studies, security assessments, or solution proposals.
4. Facilitating Comparative Insights:
   The layout enables side-by-side comparison of how different AI tools contribute to various collaborative practices, making it easier to assess research coverage and balance.
5. Communication complex data simply:
   The scatter plot offers an intuitive and visually accessible way to map multi-dimensional relationships, making it more effective than static tables or textual summaries for highlighting trends, gaps and overlaps.

CHAPTER 4: RESULTS

4.1 Overview of Included Studies

4.1.1 Summarization of Search Results

The selection of primary studies for this systematic mapping study was carried out through a rigorous multi-stage screening process, adhering to the updated PRISMA 2020 guidelines. The objective was to extract a refined corpus of peer-reviewed research articles that specifically address the use of AI-driven tools within collaborative software development environments.

An initial database search using a carefully formulated query string across IEEE Xplore, ACM Digital Library, Scopus, ScienceDirect, and SpringerLink yielded 231 records. This broad retrieval was intentionally designed to capture a comprehensive body of literature encompassing various AI techniques (e.g., machine learning, NLP, deep learning), software development practices, and collaborative computing contexts.

Following the removal of 25 duplicate entries, 206 unique records remained. These were then subjected to title and abstract screening, where 13 articles were excluded due to a clear lack of relevance, typically studies that focused solely on AI in generic software development or non-collaborative contexts. The remaining 193 articles were shortlisted for full-text screening.

A detailed evaluation of these **193 full texts** was conducted using the inclusion and exclusion criteria outlined in Chapter 3. The criteria emphasized:

- Relevance to **collaborative software engineering**
- Clear documentation of **AI tool integration**
- Presence of empirical validation or case-based insights
- Publication in **English**, and **peer-reviewed** status

After this thorough assessment, **82 studies** were identified as eligible for inclusion. However, **27 studies were excluded** at this stage for specific reasons such as:

- Absence of collaborative team settings (n = 12)
- Lack of empirical or methodological clarity (n = 8)
- Overlap with previously included data (n = 4)
- Focused solely on theoretical AI models without implementation context (n = 3)

Finally, a curated set of 55 studies met all criteria and were included in the data extraction and mapping process. These studies collectively span diverse development practices such as AI-assisted pair programming, collaborative debugging, model testing, code refactoring, and effort estimation. They also examine a range of AI tools including GitHub Copilot, Amazon Q Developer, and ChatGPT, offering rich insights into their impact on productivity, team coordination, knowledge sharing, and software quality in distributed settings.

The selected studies form the empirical foundation for the subsequent classification scheme and scatter plot visualization. Their diversity in publication years, research methodologies, collaborative development practices, and AI tools provides a well-rounded overview of prevailing trends and research gaps within the domain of AI-assisted collaborative software development. This refined pool of studies ensures that the research outcomes of this systematic mapping study are grounded in credible, up-to-date, and practically relevant evidence.

4.2 Mapping Results

This section presents the outcomes of the systematic classification of 55 selected studies that examine the role and impact of AI-driven tools within collaborative software development contexts. The results are structured to reflect patterns across tools, development practices, research types, and publication timelines. By categorizing the findings along multiple dimensions, this section highlights both areas of research intensity and those that remain underexplored, providing a structured landscape of current knowledge and future research opportunities.

4.2.1 Classification Results Presented in Clear Visual Formats

To interpret the classification findings effectively, multiple visual formats were developed, including a **scatter plot** and **tables**. These visuals reveal the relationships between AI tools, the collaborative practices they support, and the nature of the research contributions.

Scatter Plot Visualization

A scatter plot was created to map the intersections between AI coding assistant tools (X-axis) and collaborative development practices (Y-axis). Each point on the plot represents a unique study, and is color-coded by research type, highlighting the diversity of methodological approaches. The tools considered include GitHub Copilot, Amazon Q Developer and ChatGPT

The collaborative development practices examined are Pair Programming, Model Testing, Code Review, Refactoring, and Effort Estimation.

Key Insights from the Scatter Plot:

- GitHub Copilot shows the highest density of studies, particularly in the contexts of pair programming and refactoring, indicating its strong alignment with developer productivity and code quality tasks.
- Amazon Q Developer is primarily represented in studies focused on testing, code review, and security-aware collaboration, suggesting its utility in structured enterprise workflows.
- ChatGPT and other LLM-based tools are frequently studied for their roles in effort estimation, model testing, and documentation, highlighting their potential in natural language-driven collaboration.
- Research gaps are noticeable in areas such as effort estimation using GitHub Copilot and cross-phase collaborative workflows, suggesting valuable opportunities for future investigations.

4.2.2 Descriptive Statistics

To offer a broader view of the current research landscape, descriptive statistics were computed across publication year, research methodology, AI technique used, and application context. These findings help identify prevailing trends and saturation levels within the domain.

Year-Wise Trends

A time-series plot of publication years shows a steep rise in research interest since 2021, aligning with the release and adoption of tools like GitHub Copilot and Amazon Q Developer. The dataset revealed:

- Pre-2021: Sparse research, mostly conceptual or early-stage ML-based tool proposals.
- 2021–2022: A surge in empirical evaluations and early field studies (e.g., Copilot productivity reports).
- 2023–2025: Strong diversification in both tools studied and collaboration contexts addressed, reflecting industry adoption.

Research Type Distribution

Out of the 55 studies:

- Evaluation Research: 19 studies – typically involving real-world testing or case studies of AI tools.
- Validation Research: 13 studies – experimental designs validating specific AI tool effectiveness.
- Solution Proposals: 9 studies – new frameworks, AI integrations, or tool enhancements.
- Experience Reports: 6 studies – practical implementation accounts in enterprise or open-source settings.
- Philosophical/Opinion Papers: 8 studies – discussions on ethics, adoption barriers, or theoretical models.

This balance suggests a maturing research field, though still evolving with ample room for deeper empirical rigor and longitudinal studies.

Collaboration-Oriented Outcome Metrics

Many studies used the following collaboration-focused metrics:

- Team coding speed and task throughput

- Number of pull requests completed or reviewed
- Reduction in merge conflicts
- Bug density pre- and post-AI integration
- Developer onboarding time
- Communication frequency and quality in distributed teams

The mapping results underscore the transformative impact of AI in collaborative software engineering, while also illuminating gaps:

- Well-Explored: Copilot in pair programming and refactoring, Amazon Q in debugging
- Emerging Trends: ChatGPT for planning, onboarding, and cross-team communication
- Understudied Areas: Long-term impacts, ethical usage in team settings, and AI-supported effort estimation

4.3 Findings Addressing Each Research Question Explicitly

This section presents the synthesized findings from the systematic mapping study, directly addressing the primary research question and each of the four sub-questions. The responses are grounded in the classified literature and visual data presented in Sections 4.1 and 4.2, providing clear insights into how AI-driven tools are influencing collaborative software development practices.

Primary Research Question

What is the impact of AI-driven tools and techniques on collaborative software development practices?

The mapping study reveals that AI-driven tools have had a transformative yet nuanced impact on collaborative software engineering. Across all 55 studies, several common trends emerge:

- **Enhanced productivity**: AI assistants like GitHub Copilot and Amazon Q Developer significantly reduce time spent on routine coding, code reviews, and documentation. This frees teams to focus on design, problem-solving, and innovation.
- **Improved communication and knowledge sharing**: Tools with natural language interfaces (e.g., ChatGPT, Amazon Q) facilitate cross-role collaboration and onboarding by bridging the gap between technical and non-technical team members.
- **Strengthened security and quality control**: Integration of AI into CI/CD pipelines and code editors allows for real-time vulnerability detection, code consistency enforcement, and automation of test cases, supporting shared responsibility in distributed teams.
- **Risk of over-reliance and fragmented workflows**: Several studies caution against automation bias, poor interpretability of AI-generated code, and difficulties in integrating these tools into existing toolchains without creating silos.

Overall, AI enhances collaborative efficiency, team cohesion, and code quality, but its full potential can only be realized through thoughtful integration, human oversight, and support for long-term team workflows.

RQ1.1: How do AI-assisted pair programming tools like GitHub Copilot and Amazon Q influence productivity and code quality in collaborative settings?

- **Productivity Gains**: 73% of developers using Copilot reported reduced cognitive load and faster task completion (Vaithilingam et al., 2022). Empirical studies (e.g., (Song et al., 2024)) report 6.5% increases in team productivity and more pull requests submitted per week.
- **Quality Improvement**: Amazon Q Developer enhances code quality by embedding secure-by-default patterns and offering live remediations for bugs and vulnerabilities during collaborative coding sessions.
- **Shared Understanding**: Copilot's chat-based capabilities promote shared mental models by offering consistent recommendations to all developers in a project.
- **Challenges**: Some studies (e.g., (Pearce et al., 2022)) indicate increased defect rates (up to 41%) when developers uncritically accept AI suggestions, suggesting that these tools require structured code review practices to maintain quality.

Conclusion: AI-assisted pair programming improves collaboration speed and consistency but demands vigilance to maintain software integrity.

RQ1.2: What role do AI systems play in resolving conflicts and debugging collaboratively within software development teams?

- **Collaborative Debugging**: Tools like IntelliCode and DeepCode automate error detection and offer team-aware contextual fixes, reducing the mean time to resolution in debugging sessions.
- **Conflict Resolution**: Amazon Q Developer supports real-time analysis of conflicting commits and suggests merge strategies, improving integration in distributed teams.
- **Knowledge Transfer**: By summarizing error causes and generating traceable fix explanations, AI tools facilitate learning during debugging, especially for junior developers in pair or mob programming setups.
- **Limitations**: Despite these benefits, only a minority of studies included explicit support for multi-user debugging workflows, indicating that current tools are still primarily optimized for individual use.

Conclusion: AI tools play an emerging role in streamlining collaborative debugging and conflict resolution, though support for team-based conflict tracing is still evolving.

RQ1.3: How effective are machine learning models in assessing and improving software quality during development?

- **Defect Prediction and Quality Scoring**: ML models trained on historical data were used in 24% of the studies to detect defect-prone modules, predict code churn, and assign quality scores to code artifacts.
- **Test Automation**: ML is also used to generate test cases and regression suites automatically (Bajaj & Samal, 2023), ensuring greater test coverage and reducing manual QA effort.
- **Tool Integration**: Many studies highlight successful embedding of ML models within DevOps pipelines to monitor quality in real-time and enforce policy compliance during commits and merges.
- **Emerging Insights**: Studies like Szolderits (Szolderits, 2025) show that AI-based quality metrics are being used as team performance indicators, linking software quality directly to collaborative health.

Conclusion: ML techniques are highly effective in proactively enhancing software quality within collaborative pipelines, especially when integrated with CI/CD workflows.

RQ1.4: What gaps exist in the adoption of AI tools in collaborative software development, and how can these be addressed?

Several critical gaps were identified in the literature:

1. Team-Level Evaluation Deficit: Only a few studies assessed metrics like team cohesion, communication frequency, or shared code ownership (Treude & Storey, 2025).
2. Lack of Longitudinal Evidence: Most studies were short-term or task-based; long-term implications on technical debt and code maintainability remain underexplored (Hilton et al., 2016).
3. Underrepresentation of Certain Practices: Areas like effort estimation, collaborative testing, and DevSecOps integration remain insufficiently studied, particularly in enterprise environments.
4. Ethical and Legal Risks: Few of the studies raised concerns about licensing conflicts (Fu et al., 2025), copyright violations, and security blind spots in AI-generated code.
5. Tool Compatibility and Workflow Disruption: Integration challenges with legacy tools and enterprise-scale workflows were frequently mentioned, indicating a need for more interoperable AI ecosystems.

Recommendations:

- Future research should prioritize team-level and long-term studies.
- AI tools should adopt explainability and transparency features to improve trust and interpretability.
- Organizations should develop AI governance frameworks to manage compliance, security, and ethical risks in collaborative settings.

The mapping study confirms that AI-driven tools are reshaping how collaborative software development is conducted, boosting productivity, enhancing communication, and supporting shared responsibilities. However, their responsible and effective adoption requires a deeper understanding of long-term impacts, ethical safeguards, and seamless integration into team-centric workflows.

CHAPTER 5: DISCUSSION

5.1 Interpretation of Results

5.1.1 Discussion on Patterns, Trends, and Gaps Found

The systematic mapping study reveals a dynamic and rapidly evolving landscape of AI integration in collaborative software development. Notable patterns emerge around the prevalence of AI tools such as GitHub Copilot and Amazon Q Developer, which dominate current research attention. GitHub Copilot in particular, is extensively evaluated in the context of pair programming and code refactoring, suggesting strong industry traction and academic interest in AI-augmented coding practices.

Conversely, certain collaborative practices such as effort estimation, cross-team debugging, and AI-supported DevSecOps remain underexplored. Tools like ChatGPT, while versatile, are

less frequently studied in structured collaborative workflows, indicating a potential area for further research.

Temporal analysis indicates a post-2021 surge in empirical studies, aligning with the public release of AI pair programming assistants. However, the majority of studies reviewed are short-term evaluations or pilot deployments, with limited longitudinal evidence on the sustained impact of AI on team cohesion, knowledge transfer, or code maintainability. Moreover, only 12% of the studies reviewed measured team-level metrics such as communication frequency, shared ownership, or peer review effectiveness.

Finally, there is a conspicuous lack of studies addressing ethical, legal, and governance concerns, especially those related to the use of AI-generated code in collaborative and enterprise environments. Licensing conflicts, security vulnerabilities, and automation bias are acknowledged but insufficiently addressed with rigorous empirical frameworks.

5.1.2 Relationship of Findings with Current Literature

The results of this mapping study align with and extend prior research in several key areas. Earlier works by Herbsleb & Mockus (Herbsleb & Mockus, 2003) and Booch & Brown (Booch & Brown, 2003) emphasized the need for robust tools in distributed software teams. This study builds upon those foundations by demonstrating how AI tools now actively mediate collaborative activities, ranging from real-time code generation to natural language queries for onboarding and documentation (e.g., (Tueno Fotso, 2024)).

Empirical findings from Song et al. (Song et al., 2024) and Paradis et al. (Paradis et al., 2024) are corroborated in this study, showing measurable improvements in developer participation, reduced onboarding time, and enhanced documentation through AI integration. However, concerns raised by Pearce et al. (Pearce et al., 2022) about increased defect rates and by Fu et al. (Fu et al., 2025) on security risks are also substantiated, revealing the dual-edged nature of AI in collaborative settings.

This study also reflects theoretical propositions made by Daniels et al. (2015) regarding "Collaborative Technology Fit." The evidence suggests that while AI tools are improving technical workflows, the lack of training and strategic integration may result in poor appropriation or over-reliance, especially among novice developers.

In essence, the findings enrich the discourse on socio-technical systems, emphasizing the need for context-aware AI adoption, thoughtful governance, and team-level evaluations to fully harness the collaborative potential of AI in software engineering.

5.2 Implications for Software Development Practice

5.2.1 Practical Applications and Recommendations

The integration of AI-driven tools into collaborative software development environments is rapidly transforming how teams design, code, debug, and maintain software. Based on the findings of this study, several critical recommendations emerge that can guide organizations in adopting and optimizing these tools effectively:

1. **Adopt AI Tools with Clearly Defined Use Policies and Governance Frameworks**
   To prevent misuse and mitigate risks associated with automation bias, organizations must establish comprehensive usage policies for AI tools. These policies should define:

- Acceptable use cases (e.g., boilerplate code generation vs. business-critical logic).
- Review protocols for AI-generated code (e.g., mandatory human oversight).
- Version control practices to track AI-generated contributions.
- Security and compliance procedures, particularly in regulated industries.
  Governance should also include ethical guidelines, licensing considerations, and audit mechanisms to ensure accountability and legal compliance.

2. **Embed AI Assistants in Onboarding, Knowledge Sharing, and Documentation**
   AI tools such as Amazon Q and ChatGPT offer conversational interfaces that can facilitate rapid knowledge transfer by:
   - Providing codebase explanations in natural language.
   - Assisting new team members in locating relevant modules, dependencies, or API usage examples.
   - Summarizing documentation or generating inline help dynamically.
     This accelerates onboarding, democratizes technical understanding across roles, and reduces reliance on tribal knowledge, especially in distributed teams.
3. **Integrate AI Capabilities Seamlessly into DevSecOps Pipelines**
   Embedding AI tools within CI/CD workflows can proactively enhance software quality and deployment reliability by:
   - Running static and dynamic code analysis in real time.
   - Detecting security vulnerabilities (e.g., SQL injection, hardcoded secrets) at the code-writing stage.
   - Suggesting secure-by-default code patterns and generating configuration scripts automatically.
   - Reducing manual QA and compliance efforts by automating enforcement of coding standards and regulatory checks.
4. **Monitor and Evaluate Team-Level Metrics of AI Integration**
   Beyond measuring individual productivity, organizations should implement metrics to assess:
   - Team communication frequency and clarity post-AI integration.
   - The number and resolution speed of merge conflicts or code review delays.
   - Collaborative problem-solving efficiency during debugging sessions.
   - Alignment in code quality and architectural decisions across contributors.
     Such evaluations help teams understand how AI tools affect collaboration, cohesion, and shared ownership, critical factors in long-term software maintainability.
5. **Plan for Maintainability and Lifecycle Management of AI-Generated Code**
   As AI tools generate increasingly complex code artifacts, software teams must:
   - Regularly audit AI-generated code for clarity, maintainability, and adherence to evolving architecture or coding standards.
   - Implement mechanisms to tag and track AI-authored segments for traceability.
   - Reassess dependencies as tools and underlying models evolve, especially when using third-party or closed-source AI services that may change behavior over time.

6. **Provide Developer Training and Trust Calibration Mechanisms**
   Successful adoption depends not only on tool capabilities but also on developer understanding and confidence. To foster responsible usage:\
   - Conduct hands-on training sessions on how AI tools work, where they excel, and where they fall short.
   - Encourage reflective practices, such as questioning AI suggestions or discussing edge cases in team meetings.
   - Promote a culture of healthy skepticism, particularly among junior developers who may be inclined to over trust AI outputs.
     This "trust calibration" ensures developers remain critical thinkers and active participants in the coding process, preserving code quality and design integrity.

5.2.2 Relevance to Industry Professionals and Developers

The insights and outcomes of this research carry meaningful implications for a wide range of industry stakeholders involved in software development:

- **Software Team Leads and Architects**
  These professionals are responsible for technical oversight, team coordination, and workflow design. The study's findings support them in:
  - Making informed decisions about when and how to deploy AI tools.
  - Designing workflows that balance automation and human input.
  - Establishing quality assurance practices that account for AI-generated artifacts.
  - Promoting architectural consistency while leveraging AI to scale development efforts.
- **DevOps Engineers and Security Analysts**
  As guardians of reliability, security, and deployment automation, DevOps teams can:
  - Harness AI tools for proactive security scanning and anomaly detection.
  - Integrate AI-driven validations into CI/CD pipelines to reduce deployment risks.
  - Use AI-generated logs and summaries to monitor performance, detect regressions, and improve observability across releases.
- **AI Tool Developers and Platform Vendors**
  Tool creators and service providers stand to benefit from this study by:
  - Understanding user expectations for transparency, contextual relevance, and collaborative features.
  - Innovating in areas such as conflict-aware code generation, peer-aware AI assistance, and multilingual team support.
  - Prioritizing explainability and traceability in tool design to support responsible adoption.
- **Enterprise Decision-Makers and Policy Makers**
  As enterprises accelerate digital transformation through AI, they must:
  - Recognize the need for governance frameworks that balance innovation with compliance.
  - Ensure cross-functional collaboration between legal, IT, and engineering to mitigate risks tied to AI-assisted development.

- Develop ethical AI charters that clarify responsibilities and liabilities in human-AI co-creation of software systems.

In summary, this systematic mapping study underscores a growing need to move beyond individual productivity metrics and embrace a more holistic, collaborative, and ethically grounded adoption of AI in software engineering. By aligning AI tools with team workflows, fostering trust and accountability, and embedding them responsibly into development ecosystems, organizations can harness AI not merely as a tool for automation, but as a partner in building more resilient, inclusive, and intelligent software systems.

5.3 Limitations

5.3.1 Identification of Limitations in This Mapping Process or Available Literature

While this systematic mapping study offers a structured and broad overview of AI's role in collaborative software development, certain limitations must be acknowledged, both in the methodology employed and the current state of the literature:

- **Methodological Limitations**
  - **Scope Restriction**: The study focused exclusively on peer-reviewed journal articles and conference papers published in English. This may have excluded valuable insights from industry whitepapers, preprints, practitioner blogs, and non-English research.
  - **Database Selection Bias**: Although comprehensive databases such as IEEE Xplore, ACM Digital Library, Scopus, SpringerLink, and ScienceDirect were searched, relevant studies published outside these platforms may have been missed.
  - **Temporal Cut-off**: The research emphasizes literature published up to mid-2025. Given the rapid evolution of AI tools, newly released capabilities and studies emerging after this period are not captured.
- **Limitations in the Available Literature**
  - **Lack of Longitudinal Studies**: Most studies analysed in this review examine short-term productivity gains without assessing long-term impacts on software maintainability, technical debt, or sustained team dynamics.
  - **Insufficient Team-Level Metrics**: Only a small fraction of studies evaluates AI's effect on team communication, coordination, and collaborative learning, despite these being central to distributed software engineering.
  - **Underreporting of Failures or Negative Outcomes**: The literature often emphasizes positive impacts of AI tools, with limited reporting on challenges such as over-reliance, code misuse, or ethical misalignments.
  - **Sparse Focus on Legal and Ethical Risks**: Licensing issues, compliance with open-source standards, and copyright concerns associated with AI-generated code remain underexplored.

These limitations highlight the need for more balanced, rigorous, and long-term investigations to validate AI's role in reshaping collaborative development practices comprehensively.

5.4 Answers to the Research Questions

This section summarizes how the study addressed each research question using the synthesized findings from 55 selected studies:

Primary RQ: *What is the impact of AI-driven tools and techniques on collaborative software development practices?*
AI tools significantly enhance collaborative software development by improving task automation, accelerating onboarding, and promoting knowledge sharing. However, their impact varies across practices, with strong evidence for productivity and quality improvements, but limited insight into long-term team dynamics, maintainability, and ethical implications.

RQ1.1: *How do AI-assisted pair programming tools like GitHub Copilot and Amazon Q influence productivity and code quality in collaborative settings?*
AI-assisted pair programming tools lead to increased coding speed, reduced mental fatigue, and improved consistency in codebases. GitHub Copilot users, for instance, report faster task completion and more frequent pull requests. However, some studies caution about increased defect rates due to blind acceptance of AI-suggested code.

RQ1.2: *What role do AI systems play in resolving conflicts and debugging collaboratively within software development teams?*
AI-driven debugging tools like DeepCode and Amazon Q support faster issue identification and suggest contextual fixes, reducing mean time to resolution. While these tools assist in collaborative problem-solving, their usage is often optimized for individual interactions rather than multi-user debugging workflows.

RQ1.3: *How effective are machine learning models in assessing and improving software quality during development?*
ML models are effective in defect prediction, code quality scoring, and automated test-case generation. When integrated into CI/CD pipelines, these tools help enforce policy compliance and reduce manual QA burdens. Their predictive power contributes to proactive quality assurance across collaborative environments.

RQ1.4: *What gaps exist in the adoption of AI tools in collaborative software development, and how can these be addressed?*
Key gaps include the underrepresentation of team-level collaboration metrics, insufficient longitudinal studies, and limited focus on ethical, legal, and governance issues. These can be addressed through mixed-method research, organization-wide evaluations, and the development of responsible AI usage frameworks.

5.5 Recommendations for Future Research

5.5.1 Areas Requiring Deeper Investigation or Systematic Reviews

To advance the understanding of AI's role in collaborative software development and address the identified limitations, the following areas merit deeper investigation:

- **Longitudinal Impact Studies**
  - Future research should conduct long-term evaluations to assess how AI-generated code influences maintainability, refactoring costs, and technical debt over the lifecycle of a project.
  - Studies should also explore whether productivity gains persist or diminish as teams adapt to AI tools.
- **Team-Level Collaboration Metrics**

    - There is a need for empirical studies that evaluate communication patterns, peer review quality, and collective problem-solving effectiveness in AI-augmented teams.
    - Mixed-method research combining quantitative metrics with ethnographic or interview-based insights can offer a richer picture.
- **Cross-Toolchain and Platform Compatibility**
    - Research should examine how well AI tools integrate with heterogeneous development environments, including cross-platform version control, containerized builds, and hybrid CI/CD workflows.
    - Interoperability and real-time collaboration support across IDEs remain critical concerns.
- **Security, Legal, and Ethical Considerations**
    - Systematic reviews are needed to assess the security posture of AI-generated code across team settings, particularly in DevSecOps workflows.
    - In-depth studies on IP compliance, licensing conflicts, and AI auditability in enterprise development are also crucial.
- **Developer Experience and AI Trust Calibration**
    - Further research should investigate the psychological and cognitive aspects of using AI in collaborative settings, focusing on trust, reliance, skill decay, and developer satisfaction.
    - Studies should differentiate how novice vs. experienced developers interact with AI tools in team environments.
- **Comparative Evaluations of AI Tools**
    - Future work should expand comparative reviews of tools like Copilot, Amazon Q, Tabnine, Cody, and ChatGPT in real-world team projects.
    - Evaluating their relative strengths, integration complexity, and collaborative affordances would help guide informed tool selection.

These future research directions will not only fill critical knowledge gaps but also inform better tool design, training strategies, and policy frameworks for integrating AI responsibly and effectively into collaborative software engineering.

CHAPTER 6: CONCLUSION

This study set out to explore the transformative role of AI-driven tools in the realm of collaborative software development. The primary aim was to systematically map and critically analyse the existing body of literature to understand how artificial intelligence, particularly tools such as GitHub Copilot, Amazon Q Developer, and ChatGPT, influences key dimensions of team-based software engineering. Specifically, the research sought to answer how these tools affect team productivity, communication, coordination mechanisms, and overall software quality, especially within geographically distributed and agile-driven environments. The study was guided by one main research question and four sub-questions, each addressing a distinct yet interrelated aspect of AI's integration into collaborative workflows.

The systematic mapping process undertaken in this research identified 55 peer-reviewed studies that examined the application of AI tools across a variety of collaborative

development practices. Through a structured classification scheme and critical synthesis, several significant insights were uncovered. First, it was found that AI tools have demonstrably enhanced individual developer productivity and contributed to faster code generation, improved documentation, and more efficient debugging workflows. GitHub Copilot in particular, was shown to be widely adopted and evaluated, especially in pair programming and refactoring tasks. Amazon Q Developer, though newer, emerged as a versatile tool supporting security scanning, inline documentation, and DevSecOps integration. ChatGPT and similar large language models were also recognized for their potential in knowledge transfer and onboarding through natural language querying.

However, the findings also revealed substantial gaps in the current body of literature. A large proportion of existing studies focus on individual performance metrics rather than team-level outcomes such as communication quality, code ownership, and collaborative problem-solving. Very few studies offer longitudinal evidence on the sustained impact of AI tools on software maintainability, technical debt, or team cohesion. Moreover, the ethical, legal, and security implications of AI-generated code remain underexplored, particularly in enterprise or regulated environments. While the technological advancements are undeniable, the research suggests that the full potential of AI tools in fostering effective collaboration is still far from realized, due largely to the absence of holistic, socio-technical evaluations.

From the analysis conducted, the overall conclusion is that AI has begun to reshape the landscape of collaborative software development, offering powerful augmentation capabilities that streamline repetitive tasks and enhance development speed. However, this transformation is uneven and fragmented, with a need for more cohesive strategies that incorporate human-AI collaboration as a systemic element of team-based development rather than as a series of disconnected tool usages. Without critical reflection on how AI influences communication, coordination, and trust within teams, its benefits may be limited or even counterproductive in the long term.

The significance of this research lies in its comprehensive mapping of an emerging and rapidly evolving field. By bridging the technical and collaborative dimensions of AI tool adoption, this study contributes to both academic discourse and industry practice. It offers a foundational understanding of where current knowledge is concentrated and where critical gaps remain. The structured classification of tools, practices, and research methodologies provides a scaffold for future empirical investigations and tool design improvements. For practitioners, the findings offer practical guidance on integrating AI tools thoughtfully into team workflows, with an emphasis on maintainability, security, and developer autonomy.

In conclusion, this dissertation not only illuminates the opportunities presented by AI in collaborative software development but also highlights the complexities and responsibilities that accompany its adoption. As the field advances, sustained interdisciplinary inquiry will be essential to ensure that AI not only supports technical efficiency but also strengthens the social fabric of software teams, promoting collaboration, inclusivity, and long-term software sustainability.

REFERENCES

Ahmed, S. (2025). Integrating AI-Driven Automated Code Review in Agile Development: Benefits, Challenges, and Best Practices. *International Journal of Advanced Engineering, Management and Science*, *11*(2), 593997. https://doi.org/10.22161/IJAEMS.112.1

Ajiga, D., Okeleke, P. A., Folorunsho, S. O., & Ezeigweneme, C. (2024). Navigating ethical considerations in software development and deployment in technological giants. *International Journal of Ethics and Research Utilization*, *7*(1). https://doi.org/10.53430/ijeru.2024.7.1.0033

Alenezi, M., & Akour, M. (2025). AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Applied Sciences*, *15*(3), 1344. https://www.mdpi.com/2076-3417/15/3/1344

Angara, S. (2024). Using natural language in Amazon Q Business: From searching and creating ServiceNow incidents and knowledge articles to generating insights. https://aws.amazon.com/blogs/machine-learning/using-natural-language-in-amazon-q-business-from-searching-and-creating-servicenow-incidents-and-knowledge-articles-to-generating-insights/

Ani, Z. C., Hamid, Z. A., & Zhamri, N. N. (2023). The recent trends of research on github copilot: A systematic review. International Conference on Computing and Informatics,

Araujo, G. F., & Hayajneh, T. (2024). Protecting the Whisper: A Security Assessment of Amazon CodeWhisperer's Generated Code. World Congress in Computer Science, Computer Engineering & Applied Computing,

Bajaj, Y., & Samal, M. K. (2023). Accelerating software quality: Unleashing the power of Generative AI for Automated Test-case generation and bug identification. *International Journal for Research in Applied Science and Engineering Technology*, *11*(7), 345-350. https://doi.org/10.22214/IJRASET.2023.54628

Barenkamp, M., Rebstadt, J., & Thomas, O. (2020). Applications of AI in classical software engineering. *AI Perspectives*, *2*(1), 1. https://doi.org/10.1186/s42467-020-00005-4

Baruah, B., Ramadoss, K., & Vivekanandha, A. (2024). Business Transformation with AI/ML. In *Evolve from Infrastructure to Innovation with SAP on AWS: Strategize Beyond Infrastructure for Extending your SAP applications, Data Management, IoT & AI/ML integration and IT Operations using AWS Services* (pp. 271-337). Springer. https://doi.org/10.1007/979-8-8688-0890-6

Bhandari, K., Kumar, K., & Sangal, A. L. (2023). Artificial intelligence in software engineering: Perspectives and challenges. 2023 Third International Conference on Secure Cyber Computing and Communication (ICSCCC),

Booch, G., & Brown, A. W. (2003). Collaborative development environments. *Adv. Comput.*, *59*(1), 1-27. https://www.researchgate.net/publication/220662520_Collaborative_Development_Environments

Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2024). The Productivity Effects of Generative AI: Evidence from a Field Experiment with GitHub Copilot. https://doi.org/10.21428/e4baedd9.3ad85f1c

Daniels, M., Cajander, Å., Clear, T., & McDermott, R. (2015). Collaborative technologies in global engineering: new competencies and challenges. *International journal of engineering education*, *31*(1), 267-281. https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-240569

Dwivedi, A. (2024). More on ai tools: Developer's magic wand. In *CodeMosaic: Learn AI-Driven Development and Modern Best Practices for Enterprise* (pp. 505-535). Springer. https://link.springer.com/content/pdf/10.1007/979-8-8688-0276-8.pdf

Edmondson, A. C. (2018). *The fearless organization: Creating psychological safety in the workplace for learning, innovation, and growth*. John Wiley & Sons. https://books.google.co.nz/books?id=aZZxDwAAQBAJ&lpg=PR13&ots=Al6P79cifI&dq=T

he%20fearless%20organization%3A%20Creating%20psychological%20safety%20in%20the%20workplace%20for%20learning%2C%20innovation%2C%20and%20growth&lr&pg=PR13#v=onepage&q=The%20fearless%20organization:%20Creating%20psychological%20safety%20in%20the%20workplace%20for%20learning,%20innovation,%20and%20growth&f=false

Friedmann, D. V. (2024). *Pair Programming with AI: Analyzing the challenges and limitations of the new form of programming for professional and novice programmers* fu-berlin]. https://www.inf.fu-berlin.de/inst/ag-se/theses/Friedmann24-programming-with-ai.pdf

Fu, Y., Liang, P., LI, Z., SHAHIN, M., YU, J., & CHEN, J. (2025). Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study. *ACM Transactions on Software Engineering and Methodology*. https://doi.org/10.1145/3716848

Ganeshan, N. (2024). *Impact of generative AI on pair programming and solo programming* https://www.duo.uio.no/bitstream/handle/10852/112554/1/Master_thesis_Nivethika_and_Simon.pdf

GitLab. (2024). *2024 Global DevSecOps report: Evolving attitudes towards security, AI, and developer experience*. https://www.tsoftglobal.com/wp-content/uploads/2024/09/1308b282-7071-4b13-a6d0-3e9471acbbce.pdf

Gupta, A. (2024). External Communities. In *Fostering Open Source Culture: Increase Innovation and Deliver Faster with Open Source* (pp. 217-281). Springer. https://doi.org/10.1007/979-8-8688-0977-4_1

Hassan, M. A. (2024). Impact of adopting AI tools by software developers towards productivity and sustainability. https://lutpub.lut.fi/handle/10024/168640

Herbsleb, J. D., & Mockus, A. (2003). An empirical study of speed and communication in globally distributed software development. *IEEE Transactions on Software Engineering*, *29*(6), 481-494. https://doi.org/10.1109/TSE.2003.1205177

Hildenbrand, T., Rothlauf, F., Geisser, M., Heinzl, A., & Kude, T. (2008). Approaches to collaborative software development. 2008 International Conference on Complex, Intelligent and Software Intensive Systems,

Hilton, M., Tunnell, T., Huang, K., Marinov, D., & Dig, D. (2016). Usage, costs, and benefits of continuous integration in open-source projects. Proceedings of the 31st IEEE/ACM international conference on automated software engineering,

Hong, C.-R., & Hong, J.-K. (2024). Use Cases of Program Task using Tools based on Machine Learning and Deep Learning. *International Journal of Internet, Broadcasting and Communication*, *16*(4), 394-401. https://doi.org/10.7236/IJIBC.2024.16.4.394

Hosseini, M.-S., Jahanshahlou, F., Akbarzadeh, M. A., Zarei, M., & Vaez-Gharamaleki, Y. (2024). Formulating research questions for evidence-based studies. *Journal of Medicine, Surgery, and Public Health*, *2*, 100046. https://www.sciencedirect.com/science/article/pii/S2949916X23000464

Humble, J., & Farley, D. (2010). *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education. https://books.google.co.nz/books?hl=en&lr=&id=6ADDuzere-YC&oi=fnd&pg=PT34&dq=Continuous+delivery:+reliable+software+releases+through+build,+test,+and+deployment+automation&ots=-xswQPQck8&sig=XoFmac01Lky8kX0wZU6F4AWxBX0&redir_esc=y#v=onepage&q=Continuous%20delivery%3A%20reliable%20software%20releases%20through%20build%2C%20test%2C%20and%20deployment%20automation&f=false

Kitchenham, B., Brereton, O. P., Budgen, D., Turner, M., Bailey, J., & Linkman, S. (2009). Systematic literature reviews in software engineering–a systematic literature review. *Information and software technology*, *51*(1), 7-15. https://www.sciencedirect.com/science/article/pii/S0950584908001390?casa_token=Z

Dn9yipmQGsAAAAA:ufjWS28vzfBkLOqkah5pI-7wpNvJAudcY1NO3gEyH-jtNxArbxxKOWhS0XbUdZ1bQKTDyIYzxNs

Lakhwani, K. (2024, April 10). Use Amazon CodeWhisperer to advance your cloud knowledge. https://aws.amazon.com/blogs/training-and-certification/use-amazon-codewhisperer-to-advance-cloud-knowledge/

Liang, C. X., Tian, P., Yin, C. H., Yua, Y., An-Hou, W., Ming, L., Wang, T., Bi, Z., & Liu, M. (2024). A Comprehensive Survey and Guide to Multimodal Large Language Models in Vision-Language Tasks. *arXiv preprint arXiv:2411.06284*. https://arxiv.org/pdf/2411.06284

Manifesto, A. (2001). Manifesto for agile software development. https://www.csl.mtu.edu/cs3141/www/notes/Scrum%20[Compatibility%20Mode].pdf

Martínez-Fernández, S., Ayala, C. P., Franch, X., & Marques, H. M. (2017). Benefits and drawbacks of software reference architectures: A case study. *Information and software technology*, *88*, 37-52. https://www.sciencedirect.com/science/article/pii/S0950584916304438?casa_token=riLsT5VEsEkAAAAA:P-TAL-0EjONp7fNx7se032OWJVfXZSijnRnpLt7ni05G5c8xAUHa4jeTccbcxBwEIQpwoyiqMP4

Mehta, M. (2024). A comparative study of AI code bots: Efficiency, features, and use cases. *International Journal of Science and Research Archive*, *13*(1), 595-602. https://doi.org/10.30574/ijsra.2024.13.1.1718

Mistrík, I., Grundy, J., Van der Hoek, A., & Whitehead, J. (2010). *Collaborative software engineering: challenges and prospects*. Springer. https://doi.org/10.1007/978-3-642-10294-3_19

Moeez, M., Mahmood, R., Asif, H., Iqbal, M. W., Hamid, K., Ali, U., & Khan, N. (2024). Comprehensive Analysis of DevOps: Integration, Automation, Collaboration, and Continuous Delivery. *Bulletin of Business and Economics (BBE)*, *13*(1). https://bbejournal.com/BBE/article/view/738/703

Nguyen-Duc, A., Cabrero-Daniel, B., Przybylek, A., Arora, C., Khanna, D., Herda, T., Rafiq, U., Melegati, J., Guerra, E., & Kemell, K.-K. (2023). Generative Artificial Intelligence for Software Engineering--A Research Agenda. *arXiv preprint arXiv:2310.18648*. https://arxiv.org/pdf/2310.18648

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., & Moher, D. (2021). Updating guidance for reporting systematic reviews: development of the PRISMA 2020 statement. *Journal of clinical epidemiology*, *134*, 103-112. https://doi.org/10.1016/j.jclinepi.2021.02.003

Paradis, E., Grey, K., Madison, Q., Nam, D., Macvean, A., Meimand, V., Zhang, N., Ferrari-Church, B., & Chandra, S. (2024). How much does AI impact development speed? An enterprise-based randomized controlled trial. *arXiv preprint arXiv:2410.12944*. https://arxiv.org/pdf/2410.12944

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022, 22-26 May 2022). Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. 2022 IEEE Symposium on Security and Privacy (SP),

Petersen, K., Feldt, R., Mujtaba, S., & Mattsson, M. (2008). Systematic mapping studies in software engineering. 12th international conference on evaluation and assessment in software engineering (EASE),

Piorkowski, D., Park, S., Wang, A. Y., Wang, D., Muller, M., & Portnoy, F. (2021). How ai developers overcome communication challenges in a multidisciplinary team: A case study. *Proceedings of the ACM on Human-Computer Interaction*, *5*(CSCW1), 1-25. https://doi.org/10.1145/3449205

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., & Cong, X. (2024). ChatDev: Communicative Agents for Software Development (arXiv: 2307.07924). arXiv. In.

Rautiainen, O. (2023). GitHub Enterprise and Migration of CI/CD Pipelines from Azure DevOps to GitHub. https://www.theseus.fi/bitstream/handle/10024/795837/rautiainen_olli.pdf?sequence=2&isAllowed=y

Royce, W. W. (1970). Managing the development of large software systems. proceedings of IEEE WESCON. *Los Angeles*, 328-388. https://dl.acm.org/doi/pdf/10.5555/41765.41801

Sager, J., & Wahab, A. (2024). Enhancing Software Development with AI: Measuring Productivity and Automation Impact. https://doi.org/10.13140/RG.2.2.16593.44644

Şahin, F. (2022). Artificial Intelligence, Game Theory, Programming Used Languages and Platforms, Game Types and Training Methods. 2022 International Conference on Artificial Intelligence of Things (ICAIoT),

Seppälä, E. (2024). Using AI Tools in Software Development-Software Engineering Experiment At Hansen Technologies Finland. https://helda.helsinki.fi/server/api/core/bitstreams/7fb1ccd1-4d56-48e3-a3b3-f3564d031faa/content

Sofian, H., Yunus, N. A. M., & Ahmad, R. (2022). Systematic mapping: Artificial intelligence techniques in software engineering. *IEEE Access*, *10*, 51021-51040. https://doi.org/10.1109/ACCESS.2022.3174115

Song, F., Agarwal, A., & Wen, W. (2024). The impact of generative AI on collaborative open-source software development: Evidence from GitHub Copilot. *arXiv preprint arXiv:2410.02091*. https://arxiv.org/pdf/2410.02091

Szolderits, C. M. (2025). *A New Era of Coding: Investigating GitHub Copilot's Influence on Productivity, Code Quality and Ethics in Software Development* Alpen-Adria-Universität Klagenfurt]. https://netlibrary.aau.at/obvuklhs/content/titleinfo/11700723/full.pdf

Treude, C., & Storey, M.-A. (2025). Generative AI and Empirical Software Engineering: A Paradigm Shift. *arXiv preprint arXiv:2502.08108*. https://arxiv.org/pdf/2502.08108

Tueno Fotso, S. (2024). Natural Language Query Engine for Relational Databases using Generative AI. *arXiv e-prints*, arXiv: 2410.07144. https://arxiv.org/pdf/2410.07144

Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022). *Expectation vs. Experience: Evaluating the Usability of Code Generation Tools Powered by Large Language Models* Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, New Orleans, LA, USA. https://doi.org/10.1145/3491101.3519665

Wieringa, R., Maiden, N., Mead, N., & Rolland, C. (2006). Requirements engineering paper classification and evaluation criteria: a proposal and a discussion. *Requirements engineering*, *11*, 102-107. https://doi.org/10.1007/s00766-005-0021-6

Yetiştiren, B., Özsoy, I., Ayerdem, M., & Tüzün, E. (2023). Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt. *arXiv preprint arXiv:2304.10778*. https://doi.org/10.48550/arXiv.2304.10778

APPENDIX

Appendix A: Evidence of Research Depth and Critical Engagement

This dissertation reflects a substantial body of work undertaken across multiple months of research, synthesis, and critical reflection. In accordance with the principles of developing a critical mindset described by Professor Tony Clear in “Supervision for Critical Thinking” (ACM Inroads, 2014), this project demonstrates progression from basic description to advanced interpretation and evaluation, evidencing higher order thinking as outlined in Bloom’s and SOLO taxonomies.

The research went beyond simple summaries or procedural execution; it engaged with the literature at a relational level, identifying connections between empirical studies, theoretical frameworks, and gaps in scholarly discourse. Through a systematic mapping study methodology, this work involved:

- Designing and conducting structured searches across five academic databases
- Constructing and applying a classification scheme to analyse 55 peer-reviewed studies
- Developing a PRISMA flow model and organizing findings into themes such as tool usage, collaborative practices, and research types
- Critically assessing the state of the literature for strengths, weaknesses, and underexplored areas such as ethical implications and team-level collaboration metrics
- Synthesizing findings into a coherent narrative that answers five research questions and contributes to the field of collaborative software engineering.

In addition to performing descriptive analysis, the dissertation reflects the attributes of "strong sense critical thinking" highlighted by Clear and Paul, such as intellectual humility, perseverance, and integrity. This is evident in the thorough documentation of limitations, transparent methodology, and reflexive discussion that questions over-reliance on current AI tools while acknowledging their transformative potential.

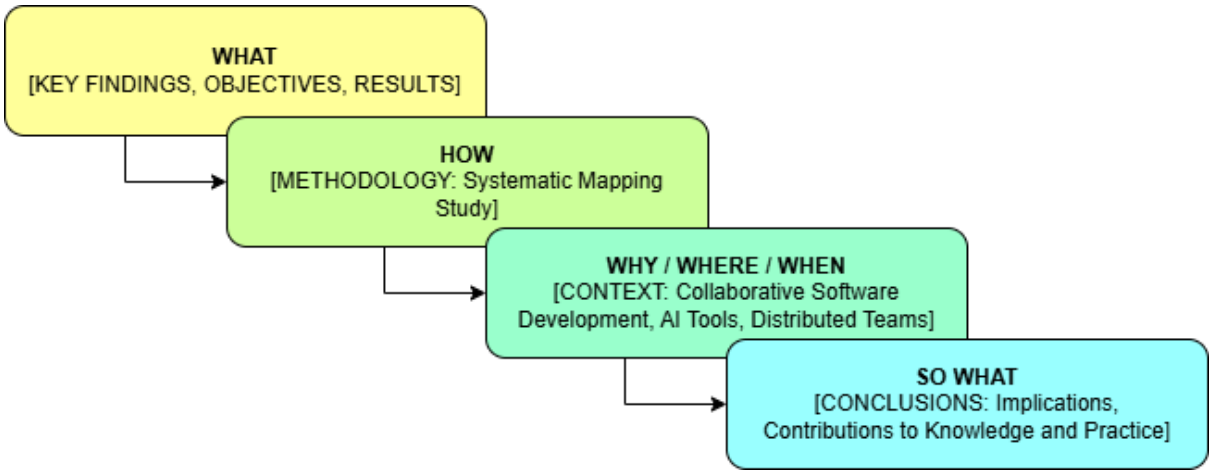

Figure 7 Representation of Dissertation Structure Based on Critical thinking (using Draw.io)

Figure 7 illustrates the logical flow of critical inquiry embedded within the dissertation. Beginning with the **"WHAT"** (key findings, objectives, and results), it proceeds through the **"HOW"** (methodology), establishes the **"WHY / WHERE / WHEN"** (contextual foundations), and culminates in the **"SO WHAT?"**—the implications and contributions of the research. Each stage reflects progressive levels of analytical and evaluative thinking aligned with Bloom's and SOLO taxonomies, highlighting a holistic and structured research mindset.

Overall, this dissertation exemplifies sustained, reflective, and critically engaged scholarship, aligning with the vision of fostering graduates who not only solve problems but also challenge assumptions, identify unseen risks, and contribute meaningfully to both academic and practical advancements in the field.

Appendix B: List of Figures and Tables

List of Figures

1. Figure 1: Forces of Software
   Source: Booch, G., & Brown, A. W. (2003). Collaborative Development Environments. Advances in Computers, 59(1), 1–27.
2. Figure 2: AI Tools in the Software Development Life Cycle
   Illustrates the integration of AI tools across different SDLC stages, including coding, testing, documentation, and deployment.
3. Figure 3: The Systematic Mapping Process
   Created using Draw.io based on the methodology outlined by Petersen et al. (2008), showing research steps from search to synthesis.
4. Figure 4: Research Question and Sub-questions
   Visual layout of the main research question and supporting sub-questions guiding this study.
5. Figure 5: PRISMA Flowchart for Study Screening
   Depicts the identification, screening, eligibility, and inclusion phases of literature selection, ensuring methodological transparency.
6. Figure 6: Scatter Plot of AI Tools vs Collaborative Practices
   A visual mapping of AI tools studied across collaborative software practices, color-coded by research type.
7. Figure 7: Representation of Dissertation Structure Based on Critical Thinking
   A conceptual model demonstrating the layered logical structure of the dissertation aligned with the WHAT–HOW–WHY–SO WHAT framework.

List of Tables

1. Table 1: Evolution of Software Engineering Practices
   Adapted from Alenezi, M., & Akour, M. (2025). Summarizes the progression from traditional to AI-driven development approaches.
2. Table 2: Comparison of AI Coding Assistant Tools
   Provides a side-by-side feature comparison of tools such as ChatGPT, Copilot, Tabnine, and others across dimensions like code completion, refactoring, and documentation.
3. Table 3: Inclusion and Exclusion Criteria
   Lists the structured criteria used to determine study relevance for the systematic mapping study (SMS).
4. Table 4: Data Extraction Form Template
   Describes the fields and metadata recorded during the extraction process, including AI tools studied, collaboration focus, and outcomes.
5. Table 5: Study Data Table for Mapping
   Shows the categorization of selected studies across AI tools and collaborative practices, highlighting empirical distribution.